

Simultaneous Classification and Solution of Linear Optimization Problems by the Dual Simplex Method: The Slope–Intercept Two-Stage Dual Simplex Method

Kan Enomoto
(Independent researcher)
<https://enomoto-math-labo.com/>

September 29, 2026

Abstract

The all-slack basis becomes dual feasible without any computation, merely by placing each nonbasic variable according to the sign of its cost coefficient, but this requires the bound at which the variable is placed to be finite. The artificial bounding method (the big- M method applied to the dual problem), which replaces infinite bounds by $\mp M$, removes this obstacle; however, as long as M is fixed as a number, there is no guarantee that M is large enough, and the method is also numerically unstable. In this paper we do not fix M ; instead, we treat the quantities arising during the iterations as affine functions of M and analyze the limit $M \rightarrow +\infty$ symbolically. We show that the iterations then split exactly into a phase that solves the *slope problem*, obtained by cutting the recession cone of the original problem with the $\|\cdot\|_\infty$ ball, and a phase that solves the *intercept problem*, obtained by fixing the slope, and that at the end of the former phase it is determined whether the original problem can have a finite optimal value. The slope problem coincides with the auxiliary problem known as Phase 1 of the dual simplex method, and this decomposition connects the artificial bounding method with Phase 1 based on the auxiliary problem. On this basis we derive the *slope–intercept two-stage dual simplex method*, which consists of the two phases followed, when necessary, by a cleanup. This method requires neither the construction of an initial basis nor a numerical value of M ; using only the dual simplex method, it simultaneously classifies the problem into the three cases (a finite optimal solution exists, the problem is unbounded, or the problem is infeasible) and, when a finite optimal solution exists, computes it. That is, classification and solution are completed within the same two phases, without a case distinction on the outcome of Phase A and without re-solving by the primal simplex method or with a modified cost. Since each phase is the dual simplex method applied to an ordinary LP, acceleration techniques for the dual simplex method, such as the steepest-edge rule and the Bound Flipping Ratio Test, can be applied as they are, and the division into phases incurs no additional computational cost.

1 Introduction

The simplex method for linear optimization (LP; also called linear programming) is a classical solution method that has been studied for many years in both theory and implementation. To actually run the simplex method, one must start from an initial basic solution that is *feasible*; that is, one must choose an invertible $m \times m$ submatrix (basis) A_B of the constraint matrix A such that, when the nonbasic variables are fixed at their upper or lower bounds, the values of the basic variables

$$x_B = A_B^{-1}(b - A_N x_N)$$

all lie within their own bounds.

Constructing such an initial basis is not trivial in general. For an *arbitrary* linear optimization problem consisting of equality constraints $Ax = b$ and general range constraints $l \leq x \leq u$ ($l_j, u_j \in \mathbb{R} \cup \{\pm\infty\}$), no general method is known that provides a feasible initial basis at no computational cost, and auxiliary procedures such as the two-phase method and the big- M method [1] have been used (Section 4).

On the other hand, if one focuses on dual feasibility, then for the all-slack basis consisting only of slack variables with cost 0, the reduced costs coincide with $\bar{c}_j = c_j$; hence, merely by placing the nonbasic original variables as

$$c_j \geq 0 \implies x_j = l_j, \quad c_j < 0 \implies x_j = u_j,$$

one obtains a dual feasible initial basis *without any computation*. This placement, however, requires the bound at which each variable is placed to be finite. In practical problems many variables have bounds that are finite on both sides, so the variables violating this requirement are often limited to a few that have an infinite side [16]. Moreover, computational experiments on real-world problems have reported that the dual simplex method was on average about twice as fast as the primal simplex method [17]. For this reason, it is of practical significance to be able to carry out, within the framework of the dual simplex method, the complete classification into “infeasible, unbounded, finite optimum” that the two-phase primal simplex method provides. In fact, major solvers sometimes return an indistinguishable verdict “infeasible or unbounded,” and re-solving with different settings is used to distinguish the two. For example, Gurobi recommends, for this verdict, re-optimizing with the dual reductions of presolve disabled [33]. Furthermore, Gurobi’s verdict “unbounded” only indicates the existence of a direction along which the objective function can be improved indefinitely, says nothing about the existence of a feasible solution, and Gurobi asks users to re-optimize with the objective function set to 0 in order to verify feasibility [33]. CPLEX has a similar verdict [34].

The artificial bounding method, which removes this obstacle by replacing infinite bounds with a large constant M , is known as one variant of Phase 1 of the dual simplex method and corresponds to the big- M method applied to the dual problem [12, 16, 18, 19]. However, as long as M is fixed as a number, the same problems as in the primal big- M method remain: there is no guarantee that M is large enough, and the introduction of M causes numerical instability. Indeed, Koberstein [16] describes an implementation that, if nonbasic variables remain at artificial bounds upon termination, multiplies M by a constant and restarts, and declares unboundedness once a threshold is exceeded; he then excludes this method from his final implementation on the grounds that an appropriate M depends strongly on the problem, that too small an M leads to repeated restarts, and that too large an M causes numerical problems.

As methods that can distinguish the three cases without requiring Phase 1, the criss-cross method [22, 23] and the self-dual parametric simplex method [1, 25] are known. Both can start from an arbitrary basis, but they combine primal and dual pivots and are not the dual simplex method itself. Consequently, implementation techniques developed for the dual simplex method, such as the steepest-edge rule and BFRT, cannot be used as they are. As for the criss-cross method, it has been stated that, in contrast to the many efficient implementations of the simplex method, no practically efficient criss-cross method is known, and its existence has been listed as an open problem [24, p. 389]. In addition, the default LP algorithms of major solvers are the dual simplex method (HiGHS, CPLEX) or a method that runs the dual simplex, interior-point and primal simplex methods concurrently (Gurobi) [26, 34, 33], and these methods are not used.

In this paper we analyze the dual simplex method applied to the bounded problem symbolically in the limit $M \rightarrow +\infty$, without fixing M as a number. We show that the iterations then split into two phases that solve the slope problem and the intercept problem in turn, and on this basis we derive a two-stage dual simplex method (the slope–intercept two-stage dual simplex method) in which the steepest-edge rule, the Devex rule and the Bound Flipping Ratio Test (BFRT) can be used as they are.

This paper is organized as follows. Section 2 distinguishes known results from the contributions of this paper; Section 3 reviews the notation of the dual simplex method with bound constraints, and Section 4 reviews known methods for constructing an initial basis. In Section 5 we bound the unbounded variables by M and give a trivial dual feasible initial basis. In Section 6 we analyze the behavior as $M \rightarrow +\infty$ symbolically and establish the classification of the original problem into the three cases, the cleanup, the non-return to infinite bounds, and anti-cycling. In Section 7 we show that the iterations split into two phases that solve the slope problem and the intercept problem and compare this with existing two-phase dual simplex methods, and in Section 8 we interpret this from the side of the dual problem. Section 9 summarizes the slope–intercept two-stage dual simplex method, and Section 10 reports the results of numerical experiments.

Contents

1	Introduction	1
2	Known Results and Contributions of This Paper	4
3	Preliminaries: The Dual Simplex Method with Bound Constraints	5
3.1	The dual simplex method	5
4	Construction of an Initial Basis	6
5	Bounding by M and a Trivial Dual Feasible Basis	7
5.1	Preprocessing: shift of variables with a single infinite bound	7
5.2	Bounding the unbounded variables by M	7
5.3	A trivial initial basis via zero-fixed slack variables	7
5.4	The algorithm with M fixed	8
6	Symbolic analysis as $M \rightarrow +\infty$	10
6.1	Affinity of the basic solution and lexicographic comparison	10
6.2	Classification into the three cases	11
6.3	Cleanup of nonbasic variables remaining at artificial bounds	13
6.4	Non-return to infinite bounds and interpretation of the method	15
6.5	Anti-cycling: extending Bland’s rule to the lexicographic order	17
7	Decomposition into the slope problem and the intercept problem	18
7.1	The slope problem	18
7.2	Two-phase decomposition	20
7.3	The intercept problem	21
7.4	Overall picture: a chain of problems solved while passing on the basis	22
7.5	Comparison with existing two-phase dual simplex methods	24
8	Interpretation from the dual problem	26
8.1	The big- M method for the dual problem	26
8.2	The two-phase method for the dual problem	27
9	The slope–intercept two-stage dual simplex method: summary	28
10	Numerical experiments	32
10.1	Overview of the implementation	32
10.2	Implementation techniques not found in HiGHS	33
10.3	Experimental environment and measurement	33

10.4 Results on Netlib	34
10.5 Comparison on large-scale problems	34

2 Known Results and Contributions of This Paper

Since the content of this paper mixes a reorganization of known results with new theory, we make the distinction between the two explicit here.

Known results. The following items are known or follow immediately from known frameworks. In this paper we use them as the foundation of our discussion and, where needed, give proofs in our setting, but we do not regard them as main contributions.

- The dual simplex method with bound constraints including status Z , in which a free variable is kept nonbasic at value 0 [3, 7] (Section 3), and the two-phase method, the big- M method, crash procedures and Phase 1 of the dual simplex method for constructing an initial basis (Section 4).
- The artificial bounding method, which replaces infinite bounds with $\pm M$ (Section 5.2). This is the dual version of the composite simplex method [12], that is, the big- M method applied to the dual problem [16, 18, 19]. Restoring the original bounds of a variable given artificial bounds once it enters the basis, flipping variables with finite bounds on both sides, and combining the method with BFRT have also already been described for implementations using a numerical M [16].
- The idea of treating M as a non-Archimedean infinity and executing the big- M method by lexicographic comparison of coefficients [10, 11], and the fact that the big- M method coincides with the two-phase method as $M \rightarrow +\infty$ [8, 9]. Lemma 6.1, Propositions 6.3 and 6.5, and the interpretation in Section 8 correspond to transferring these to the side of the dual problem.
- The primal push of crossover, which moves superbasic variables with reduced cost 0 to their bounds [20, 21, 37]. The cleanup in Lemma 6.6 has the same structure (Remark 6.9).
- Anti-cycling by a lexicographic rule for the ratio test [2] (Section 6.5), and the characterization of boundedness by the sign of the objective function on the recession cone [9] (Corollary 7.3).
- The slope problem itself. It coincides with the auxiliary problem of minimizing the sum of dual infeasibilities, known as Phase 1 of the dual simplex method [13, 14, 15, 16, 18] (Section 7.1).

Contributions of this paper. The contribution of this paper is to treat M in the artificial bounding method symbolically rather than fixing it as a number, to clarify the structure of its iterations, and then to put it into shape as a practical dual simplex method.

- (1) **Simultaneous classification and solution by the dual simplex method** (Propositions 6.5 and 9.1): starting from the all-slack basis obtained without computation, and using only iterations of the dual simplex method, we determine exactly whether the original problem has a finite optimal value, is feasible and unbounded, or is infeasible, and, when it has a finite optimal value, we construct an optimal basic solution. Classification and solution are not separate procedures: they are carried out simultaneously within the same two phases, with no case distinction on the outcome of Phase A and no re-solving of a different problem. The artificial bounding method with a numerical M cannot distinguish an insufficient M from unboundedness and relies on heuristic decisions, while

existing two-phase dual simplex methods based on the auxiliary problem stop as soon as dual infeasibility is detected and do not distinguish infeasibility from unboundedness [16] (Section 7.5).

- (2) **Decomposition into slope and intercept** (Propositions 7.2 and 7.4): the two components of the lexicographic comparison never act simultaneously in the main iterations, and the iterations split, at the level of individual iterations, into Phase A, which is the dual simplex method applied to the slope problem, and Phase B, which is the classical dual simplex method applied to the intercept problem obtained by fixing the slope. Because the slope problem appears as the coefficient of M in the artificial bounding method, the artificial bounding method (the big- M method applied to the dual problem) and Phase 1 based on the auxiliary problem (the two-phase method applied to the dual problem), which have so far been treated as separate techniques, are connected (Section 8.2). To the best of our knowledge, no literature has made this correspondence explicit.
- (3) **The slope–intercept two-stage dual simplex method** (Sections 7.5 and 9): we give a two-stage algorithm that, after solving in Phase A the same problem as the existing Phase 1, solves the intercept problem rather than the original problem. By solving the intercept problem directly from the basis of Phase A regardless of the sign of z^1 , the same procedure solves the problem and detects infeasibility when $z^1 = 0$, and distinguishes infeasibility from unboundedness when $z^1 < 0$. In the case $z^1 = 0$ where variables remain at artificial bounds, we give, apart from the existing method of re-placing them at their finite sides, a route that obtains an optimal basic solution by the intercept problem and the cleanup while retaining x^1 , and we understand both in a unified way as differing only in how variables with reduced cost 0 are placed. The main iterations do not use the lexicographic comparison, and each phase is the dual simplex method applied to an ordinary LP; hence correctness is shown directly (Proposition 9.1), and the steepest-edge rule, the Devex rule and BFRT can be applied as they are.
- (4) **Symbolic cleanup** (Lemma 6.6): we execute the primal push lexicographically in a setting where the step lengths and the basic solution are affine functions of M , and construct an optimal basic solution of the original problem without ever fixing M as a number.

On the other hand, the selection rules for the leaving row and the entering variable are essentially unchanged by the separation (Remark 7.7), and a comparison of efficiency with existing methods is addressed through the numerical experiments in Section 10.

3 Preliminaries: The Dual Simplex Method with Bound Constraints

In this paper we consider linear optimization problems of the following form:

$$\begin{aligned} \min \quad & c^\top x \\ \text{s.t.} \quad & Ax = b, \\ & l_j \leq x_j \leq u_j, \quad j = 1, \dots, n, \end{aligned} \tag{1}$$

where $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, $c \in \mathbb{R}^n$, $\text{rank}(A) = m$, and $l_j \in \mathbb{R} \cup \{-\infty\}$, $u_j \in \mathbb{R} \cup \{+\infty\}$, $l_j \leq u_j$.

3.1 The dual simplex method

We fix the notation used below for the dual simplex method with bound constraints [3, 7, 16]. The index set is partitioned into a basis B ($|B| = m$, A_B nonsingular) and a nonbasis N ; each nonbasic variable is assigned a status $\eta(j) \in \{L, U, Z\}$, and $x_j = l_j, u_j, 0$ according as

$\eta(j) = L, U, Z$. The statuses L, U denote placement at a finite lower or upper bound, and status Z keeps a free variable nonbasic at value 0 [7]. The basic variables are determined by $x_B = A_B^{-1}(b - A_N x_N)$, and (B, η) is said to be *primal feasible* if $l_B \leq x_B \leq u_B$. With the simplex multipliers $y^\top = c_B^\top A_B^{-1}$ and the reduced costs $\bar{c}_j = c_j - y^\top A_j$, (B, η) is said to be *dual feasible* if for all $j \in N$

$$\eta(j) = L \implies \bar{c}_j \geq 0, \quad \eta(j) = U \implies \bar{c}_j \leq 0, \quad \eta(j) = Z \implies \bar{c}_j = 0$$

holds. A basis that is both primal and dual feasible is an optimal basis.

One iteration starting from a dual feasible basis is as follows.

- (i) **Leaving row:** choose a row r that violates its bounds, and let $\beta(r)$ be the index of its basic variable. Set $\rho = +1$ if $x_{B,r} < l_{\beta(r)}$ and $\rho = -1$ if $x_{B,r} > u_{\beta(r)}$. Since $x_{B,r}$ is finite, the bound of $\beta(r)$ on the violated side is necessarily finite.
- (ii) **Pivot row:** let $\alpha^\top := e_r^\top A_B^{-1} A_N$, define the sign σ_j to be $+1$ if $\eta(j) = L$, -1 if $\eta(j) = U$, and the sign opposite to $\rho \alpha_j$ if $\eta(j) = Z$, and set $\hat{\alpha}_j := \sigma_j \alpha_j$, $\hat{c}_j := \sigma_j \bar{c}_j$ (≥ 0).
- (iii) **Ratio test:** let $\text{Eligible} := \{j \in N : l_j < u_j, \rho \hat{\alpha}_j < 0\}$ (for the reason for excluding fixed variables, see Remark 5.2). If $\text{Eligible} = \emptyset$, the problem is infeasible and we stop. Otherwise, take $q \in \arg \min_{j \in \text{Eligible}} \hat{c}_j / |\hat{\alpha}_j|$ as the entering variable. Candidates with status Z have $\hat{c}_j = 0$ and are therefore given top priority with ratio 0.
- (iv) **Pivot:** fix $\beta(r)$ at the bound on its violated side, making it nonbasic, and make q basic.

When all rows satisfy their bounds, an optimal basis has been obtained, and under an anti-cycling rule the iterations terminate after finitely many steps [7, 16].

Remark 3.1. The reduced cost of a nonbasic variable k with status Z remains 0 throughout the iterations. This is because, in the update $\bar{c}_k \leftarrow \bar{c}_k - \theta \alpha_k$ (θ being the dual step length), if $\alpha_k \neq 0$ then k can enter with ratio 0, so that $\theta = 0$.

4 Construction of an Initial Basis

A primal feasible initial basis is not trivial in general, and the two-phase method, the big- M method [1] and crash procedures [7] are used. On the other hand, for the all-slack basis consisting of slack variables with cost 0, we have $y = 0$ and hence $\bar{c}_j = c_j$, so dual feasibility is achieved merely by setting $\eta(j) = L$ if $c_j \geq 0$ and $\eta(j) = U$ if $c_j < 0$. This placement, however, requires the bound at which each variable is placed to be finite, which does not hold in general except in cases such as bounded-variable problems in which all variables have finite bounds on both sides. Methods that remove this obstacle are known as Phase 1 of the dual simplex method; they include minimization of the sum of dual infeasibilities, the cost shifting method, and the artificial bounding method (the big- M method applied to the dual problem), which replaces infinite bounds with a large constant [12, 7, 16, 18, 19].

The numerical difficulty of the big- M method stems from fixing M as a concrete number. For a fixed basic solution, the objective function of the big- M method is an affine function $z^1 M + z^0$ of M , so comparison for sufficiently large M reduces to lexicographic comparison of the coefficient pairs (z^1, z^0) , and the big- M method is equivalent to lexicographic (preemptive) optimization [10]. Cococcioni and Fiaschi [11] treat M as a non-Archimedean infinite number and realize this idea as an algorithm for the primal big- M method. The point of this paper is to apply this idea to the big- M method applied to the dual problem, that is, to M as a *bound* on variables.

5 Bounding by M and a Trivial Dual Feasible Basis

5.1 Preprocessing: shift of variables with a single infinite bound

All infinite bounds, including those of free variables, are handled directly by the bounding by M in the next subsection, and we do not perform preprocessing that eliminates free variables from the equality constraints. To simplify the subsequent discussion, we perform the following preprocessing beforehand. For each $j = 1, \dots, n$, define the shift amount

$$t_j := \begin{cases} u_j & (l_j = -\infty, u_j < +\infty) \\ l_j & (l_j > -\infty, u_j = +\infty) \\ 0 & (\text{otherwise, i.e., both sides finite or both sides infinite (free variable)}) \end{cases}$$

(to avoid confusion with the slack variables $s \in \mathbb{R}^m$, we denote the shift amounts by $t \in \mathbb{R}^n$), and substitute $x_j = x'_j + t_j$. The constraints then become $Ax' = b - At =: b'$, and the bounds become $l'_j := l_j - t_j$, $u'_j := u_j - t_j$. In particular, for each j that had exactly one infinite bound, the bound on the finite side becomes exactly 0 (for example, if $l_j = -\infty, u_j < \infty$ then $l'_j = -\infty, u'_j = 0$; if $l_j > -\infty, u_j = +\infty$ then $l'_j = 0, u'_j = +\infty$). For free variables $t_j = 0$, and $l'_j = -\infty, u'_j = +\infty$ remain unchanged by the shift. Since the objective function becomes $c^\top x = c^\top x' + c^\top t$, the optimal value of the original problem is the optimal value of the shifted problem plus the constant $c^\top t$.

In what follows we assume that this preprocessing has been done, and we use the symbols b, l, u to mean b', l', u' . That is, we may assume that, for variables with a single infinite bound, the bound on the finite side is 0.

5.2 Bounding the unbounded variables by M

Fix an arbitrary $M > 0$. For each $j = 1, \dots, n$, define the bounded bounds

$$\hat{l}_j := \begin{cases} l_j & (l_j > -\infty) \\ -M & (l_j = -\infty) \end{cases}, \quad \hat{u}_j := \begin{cases} u_j & (u_j < +\infty) \\ M & (u_j = +\infty) \end{cases} \quad (2)$$

Bounds that were originally finite are used as they are, and only the infinite sides are replaced by $\mp M$. Since all $\hat{l}_j, \hat{u}_j$ are finite, the problem

$$\min c^\top x \quad \text{s.t.} \quad Ax = b, \quad \hat{l} \leq x \leq \hat{u} \quad (3)$$

is a bounded problem to which the simplex method with bound constraints applies directly. Clearly $[\hat{l}, \hat{u}] \subseteq [l, u]$, so the feasible region of (3) is a subset of the feasible region of the original problem (1). Moreover, if $M_1 \leq M_2$ then $[\hat{l}(M_1), \hat{u}(M_1)] \subseteq [\hat{l}(M_2), \hat{u}(M_2)]$, and the feasible region of (3) is monotonically nondecreasing in M .

5.3 A trivial initial basis via zero-fixed slack variables

We add to the equality constraints $Ax = b$ slack variables $s \in \mathbb{R}^m$, one for each row, and rewrite them as

$$Ax + s = b, \quad l_s = u_s = 0 \quad (\text{i.e., } s \equiv 0 \text{ is fixed}).$$

The extended variable vector is $(x, s) \in \mathbb{R}^{n+m}$, and the extended constraint matrix is $\hat{A} = [A \mid I_m]$. The cost of s is 0.

If we now adopt $B_0 := \{n+1, \dots, n+m\}$ (the index set of the slack variables) as the initial basis, then $A_{B_0} = I_m$, so

$$A_{B_0}^{-1} = I_m$$

and not even the computation of an inverse is required. The statuses of the nonbasic (original) variables are determined solely from the signs of the cost coefficients as

$$\eta(j) = \begin{cases} L, & x_j = \hat{l}_j & c_j > 0, \text{ or } (c_j = 0 \text{ and } l_j > -\infty) \\ U, & x_j = \hat{u}_j & c_j < 0, \text{ or } (c_j = 0 \text{ and } l_j = -\infty, u_j < +\infty) \\ Z, & x_j = 0 & c_j = 0 \text{ and } l_j = -\infty, u_j = +\infty \end{cases} \quad (j = 1, \dots, n). \quad (4)$$

A variable with cost 0 satisfies the dual feasibility condition whichever of L, U, Z it is placed at, so we choose a placement whose value does not depend on M . That is, if there is a finite side, the variable is placed there, and a free variable is placed at value 0 with status Z . In this way, the only variables whose initial values depend on M are those belonging to the set of variables for which the sign of the cost strictly requires placement on an infinite side,

$$S := \{j : l_j = -\infty, c_j > 0\} \cup \{j : u_j = +\infty, c_j < 0\}.$$

The values of the basic (slack) variables are then computed directly by

$$x_{B_0} = s = b - Ax_N. \quad (5)$$

In general s does not coincide with 0, and the value in (5) may violate the bounds $[0, 0]$ of s itself; as we shall see below, however, this poses no problem from the standpoint of dual feasibility.

Proposition 5.1 (Dual feasibility of the initial basis). *The pair (B_0, η) defined by (4) is dual feasible for problem (3).*

Proof. The basic costs are $c_{B_0} = 0$ (the cost of s is 0) and $A_{B_0} = I$, so the simplex multipliers are $y^\top = c_{B_0}^\top A_{B_0}^{-1} = 0$. Hence the reduced cost of a nonbasic original variable j is

$$\bar{c}_j = c_j - y^\top A_j = c_j,$$

which coincides with the cost itself. By (4), if $\eta(j) = L$ then $c_j > 0$ or $(c_j = 0 \text{ and } l_j > -\infty)$; in either case $c_j \geq 0$, hence $\bar{c}_j = c_j \geq 0$. Similarly, if $\eta(j) = U$ then $c_j < 0$ or $(c_j = 0 \text{ and } l_j = -\infty)$; in either case $c_j \leq 0$, hence $\bar{c}_j = c_j \leq 0$. If $\eta(j) = Z$ then $c_j = 0$, so $\bar{c}_j = 0$. Therefore the dual feasibility conditions are satisfied. The reduced costs of the basic variables (s) are identically 0 by definition, and no condition needs to be imposed on them. $\square$

What is essentially used here are two points: (a) since $c_{B_0} = 0$ and $A_{B_0} = I$, the simplex multipliers are trivially $y = 0$; and (b) the bounds $\hat{l}_j, \hat{u}_j$ at which variables are placed are all finite. Point (b) is guaranteed by the bounding by M .

5.4 The algorithm with M fixed

From the above, the algorithm for problem (3), bounded with M fixed, is as follows.

Step 1 (Construction of the initial basis). By (4) and (5), set the basis $B \leftarrow B_0 = \{n+1, \dots, n+m\}$ (the indices of the slack variables), the statuses of the nonbasic original variables to η , the nonbasic values to x_N , and the basic values to $x_B = s = b - Ax_N$. By Proposition 5.1, this (B, η) is dual feasible without any computation.

Step 2 (Iteration). Repeat the following (a)–(d) until termination is determined in (a).

- (a) **Test of primal feasibility (termination condition).** The values of the basic variables are $x_B = A_B^{-1}(b - A_N x_N)$, and the nonbasic values are $x_j = \hat{l}_j, \hat{u}_j, 0$ according as $\eta(j) = L, U, Z$ (in the first pass these coincide with (5), and thereafter they may be updated successively by the pivots in (d)). In particular, if originally $l_j = -\infty$ and $\eta(j) = L$ then

$x_j = -M$, and if originally $u_j = +\infty$ and $\eta(j) = U$ then $x_j = M$. Let $\beta(i)$ be the index of the basic variable corresponding to row i , and define the set of violating rows

$$V := \{ i \in \{1, \dots, m\} : x_{B,i} < \hat{l}_{\beta(i)} \text{ or } x_{B,i} > \hat{u}_{\beta(i)} \}.$$

If $V = \emptyset$, then (B, η, x) is primal feasible for problem (3); since dual feasibility holds in Step 1 and is preserved in (b)–(d) (Section 3.1), it is an optimal solution of problem (3). The algorithm terminates. If $V \neq \emptyset$, proceed to (b).

- (b) **Selection of the leaving variable.** First, for each $i \in V$, define the violation

$$\Delta_i := \max(\hat{l}_{\beta(i)} - x_{B,i}, x_{B,i} - \hat{u}_{\beta(i)}) (> 0).$$

The leaving row $r \in V$ is chosen by the largest-violation rule (Dantzig's rule) $r \in \arg \max_{i \in V} \Delta_i$, or by the steepest-edge rule [5], which maximizes Δ_i^2 / γ_i normalized by the row weights $\gamma_i := \sum_{j \in N} \alpha_{ij}^2$ ($\alpha_{ij} := (A_B^{-1} A_N)_{ij}$), or by the Devex rule [6], which replaces γ_i with approximate weights $\tilde{\gamma}_i$ that are cheap to update (the $M \rightarrow \infty$ version of Dantzig's rule is given in Section 6.1, and the steepest-edge and Devex rules are used as they are for the problem of each phase in the final algorithm of Section 9). The direction of violation ρ is determined with respect to $\hat{l}, \hat{u}$ as in (i) of Section 3.1.

- (c) **Computation of the pivot row and ratio test.** As in (ii) of Section 3.1, define $\alpha^\top := e_r^\top A_B^{-1} A_N$, $\sigma_j, \hat{\alpha}_j := \sigma_j \alpha_j$, $\hat{c}_j := \sigma_j \bar{c}_j (\geq 0)$, and compute the set of nonbasic variables eligible to enter,

$$\text{Eligible} := \{ j \in N : \hat{l}_j < \hat{u}_j, \rho \hat{\alpha}_j < 0 \}$$

(fixed variables are excluded; see Remark 5.2 below). If $\text{Eligible} = \emptyset$, problem (3) is infeasible under the current M , and the algorithm terminates. Otherwise, choose the entering variable q by

$$q \in \arg \min_{j \in \text{Eligible}} \frac{\hat{c}_j}{|\hat{\alpha}_j|}$$

(candidates with status Z are given top priority with ratio 0).

- (d) **Pivot.** Make $\beta(r)$ nonbasic, fixing it at $\hat{l}_{\beta(r)}$ if $\rho = +1$ and at $\hat{u}_{\beta(r)}$ if $\rho = -1$. Make q basic, update the basis B , the inverse A_B^{-1} , the basic solution x_B , the nonbasic values x_N and the nonbasic statuses η , and return to (a).

Remark 5.2 (Exclusion of fixed variables). A nonbasic variable with $\hat{l}_j = \hat{u}_j$ (a fixed variable) satisfies dual feasibility without changing its value, merely by switching its status between L and U according to the sign of its reduced cost; hence it may be excluded from the ratio test [16]. In particular, since the slack variables s have $l_s = u_s = 0$, a slack variable that has once left the basis never returns to it.

Running this algorithm with a numerical M fixed gives rise to two problems. First, the nonbasic values $\mp M$ enter the computation of x_B , the test of violations and the ratio test, causing numerical instability of the same kind as in the classical big- M method. Second, and more fundamentally, unless M is guaranteed to be large enough, the verdict itself may be wrong. Even if the original problem is unbounded, if M is too small then (3) becomes a bounded problem in which the unbounded directions are cut off, and a finite optimal solution is returned. In the next section we avoid these problems by not fixing the value of M and analyzing the behavior as $M \rightarrow +\infty$ symbolically.

6 Symbolic analysis as $M \rightarrow +\infty$

6.1 Affinity of the basic solution and lexicographic comparison

As noted in Section 4, in the big- M method a comparison for sufficiently large M reduces to a lexicographic comparison of coefficient pairs. Since M in this paper plays the role of a penalty coefficient in the dual problem (Section 8), the limit $M \rightarrow +\infty$ can likewise be handled as a lexicographic comparison of coefficients.

Steps 2(c)(d) are determined solely by the current basis B and the status η of the nonbasic variables, and do not depend on the numerical value of M . M enters only through $\hat{l}, \hat{u}$, through x_N, x_B determined by them, and through the tests and selections in Steps 2(a)(b). In what follows, we fix the combinatorial state (B, η) and carry out the analysis using the fact that these quantities are affine functions of M .

Lemma 6.1 (Affinity of x_B in M). *Fix the combinatorial state (B, η) . Each component of the nonbasic values $x_N(M)$ is*

$$x_j(M) = \begin{cases} l_j & (\eta(j) = L, l_j > -\infty) \\ -M & (\eta(j) = L, l_j = -\infty) \\ u_j & (\eta(j) = U, u_j < +\infty) \\ M & (\eta(j) = U, u_j = +\infty) \\ 0 & (\eta(j) = Z) \end{cases}$$

so we can write $x_N(M) = x_N^0 + Mx_N^1$. Here $x_N^0 \in \mathbb{R}^{|N|}$ and $x_N^1 \in \{-1, 0, 1\}^{|N|}$ are constant vectors that depend only on (B, η) and not on M , with $x_j^1 = -1$ ($\eta(j) = L, l_j = -\infty$), $x_j^1 = +1$ ($\eta(j) = U, u_j = +\infty$), and $x_j^1 = 0$ otherwise (including status Z). Hence

$$x_B(M) = A_B^{-1}(b - A_N x_N(M)) = \underbrace{A_B^{-1}(b - A_N x_N^0)}_{=: x_B^0} + M \underbrace{(-A_B^{-1} A_N x_N^1)}_{=: x_B^1}$$

is an affine function of M , and the coefficients $x_B^0, x_B^1 \in \mathbb{R}^m$ depend only on (B, η) . That is, setting $x^0 := (x_N^0, x_B^0)$ and $x^1 := (x_N^1, x_B^1)$, we have $x(M) = x^0 + Mx^1$.

Throughout what follows, for an affine function $f(M)$ of M we denote its *slope* (the coefficient of M) by a superscript 1 and its *intercept* (the constant term) by a superscript 0, and write $f(M) = f^1 M + f^0$. The basic solution $x(M) = x^0 + Mx^1$ follows this notation.

Similarly, by (2), $\hat{l}_{\beta(i)}(M)$ and $\hat{u}_{\beta(i)}(M)$ are the affine functions ($l_{\beta(i)}$ or $-M$) and ($u_{\beta(i)}$ or M), respectively, so by Lemma 6.1 the violations in Step 2(a),

$$v_i^-(M) := \hat{l}_{\beta(i)}(M) - x_{B,i}(M), \quad v_i^+(M) := x_{B,i}(M) - \hat{u}_{\beta(i)}(M),$$

are also affine functions of M . Following the notation above, we write them as $v_i^-(M) = v_i^{-,1}M + v_i^{-,0}$ and $v_i^+(M) = v_i^{+,1}M + v_i^{+,0}$ (where $v_i^{\pm,1}, v_i^{\pm,0} \in \mathbb{R}$ are constants depending only on (B, η)).

Definition 6.2 (Lexicographic order on affine functions). For two affine functions $f(M) = f^1 M + f^0$ and $h(M) = h^1 M + h^0$, define

$$f \succ h \quad :\Longleftrightarrow \quad f^1 > h^1, \text{ or } (f^1 = h^1 \text{ and } f^0 > h^0)$$

(that is, we compare the coefficient vectors $(f^1, f^0), (h^1, h^0)$ in lexicographic order).

If $f \succ h$, then there exists $M^\dagger(f, h) > 0$ such that $f(M) > h(M)$ for all $M \geq M^\dagger(f, h)$ (if $f^1 \neq h^1$ one may take $M^\dagger = |f^0 - h^0|/|f^1 - h^1|$, and if $f^1 = h^1$, $f^0 > h^0$ the inequality holds for every $M > 0$). The case $h \succ f$ is analogous, and when $(f^1, f^0) = (h^1, h^0)$ we have $f \equiv h$. This $\succ$ gives a total order on any finite set of affine functions, and if $M^\dagger$ denotes the maximum of $M^\dagger(f, h)$ over all pairs among them, then as long as $M \geq M^\dagger$, the outcomes of all numerical comparisons coincide with the comparisons by $\succ$.

Using this, Steps 2(a)(b) can be rewritten as follows without fixing a specific value of M .

(a') **Primal feasibility test (termination condition, $M \rightarrow \infty$ version).** Let $V_\infty := \{i \in \{1, \dots, m\} : v_i^- \succ 0 \text{ or } v_i^+ \succ 0\}$ (where 0 denotes the affine function that is identically 0, i.e., the coefficient pair $(0, 0)$). If $V_\infty = \emptyset$, primal feasibility holds for all $M \geq M^\dagger$, and the algorithm terminates.

(b') **Selection of the leaving variable (Dantzig rule, $M \rightarrow \infty$ version).** For each $i \in V_\infty$, let w_i be the larger of v_i^-, v_i^+ with respect to $\succ$. Choose the leaving row r by

$$r \in \arg \max_{i \in V_\infty} \succ w_i$$

(as a maximal element with respect to the lexicographic order $\succ$). Set $\rho = +1$ if $w_r = v_r^-$ and $\rho = -1$ if $w_r = v_r^+$ (ties, when $\arg \max \succ$ contains several rows, are broken deterministically by a rule in the spirit of Bland's rule, such as giving priority to the smaller index).

Proposition 6.3 (Agreement with numerical computation for sufficiently large M). *For a fixed combinatorial state (B, η) , there exists $M^\dagger > 0$ (depending only on that state) such that the tests and selections by (a')(b') coincide with the tests and selections by the numerical computation in (a)(b) for all $M \geq M^\dagger$.*

Proof. The membership test for each element of V_∞ and the choice of r by $\arg \max \succ$ are determined solely by comparisons with respect to $\succ$ among the finitely many affine functions v_i^-, v_i^+ and 0. As noted above, beyond some common $M^\dagger$ these comparisons coincide with the numerical comparisons $v_i^\pm(M) > 0$ and $\max_i(\cdot)$. $\square$

Since Steps 2(c)(d) do not depend on M , (a')(b') uniquely determine the next state (B', η') (without fixing M as a number), and repeating this yields a sequence of combinatorial state transitions $(B^{(0)}, \eta^{(0)}) \rightarrow (B^{(1)}, \eta^{(1)}) \rightarrow \dots$. Under an anti-cycling rule (Section 6.5) this sequence terminates after finitely many steps, and if we redefine $M^\dagger$ as the maximum of the $M^\dagger$ over all iterations, then for every $M \geq M^\dagger$ the sequence of iterates obtained by numerically computing Step 2 coincides exactly with this combinatorial transition sequence. In other words, the behavior as $M \rightarrow +\infty$ can be computed solely by lexicographic comparisons of coefficient pairs. The final state (B^*, η^*) of this transition sequence is itself an optimal basis of (3) for sufficiently large M . Hereafter, we call this iteration in which M is treated symbolically (Steps 2(a')(b')(c)(d)) the *symbolic artificial bounding method*.

6.2 Classification into the three cases

Suppose the state transition sequence has reached the final state (B^*, η^*) (a state in which $V_\infty = \emptyset$ in (a')). Using $x(M) = x^0 + Mx^1$ from Lemma 6.1, the optimal value of problem (3) is again expressed as an affine function of M :

$$z(M) = c_B^\top x_B(M) + c_N^\top x_N(M) = \underbrace{c^\top x^0}_{=: z^0} + M \underbrace{c^\top x^1}_{=: z^1}.$$

Here $z^0, z^1 \in \mathbb{R}$ depend only on (B^*, η^*) , and for $M \geq M^\dagger$ this is exactly the optimal value of problem (3). Since the feasible region of (3) is monotonically nondecreasing in M (Section 5.2), $z(M)$ is monotonically nonincreasing, and hence $z^1 \leq 0$ always holds.

Lemma 6.4 (Union of feasible regions). *Let $F := \{x : Ax = b, l \leq x \leq u\}$ (the feasible region of the original problem (1)). Then*

$$F = \bigcup_{M>0} F_M, \quad F_M := \{x : Ax = b, \hat{l}(M) \leq x \leq \hat{u}(M)\} \text{ (i.e., the feasible region of (3))}$$

holds.

Proof. ($\supseteq$): For any $M > 0$ we have $[\hat{l}(M), \hat{u}(M)] \subseteq [l, u]$, so $F_M \subseteq F$.

($\subseteq$): Take any $x \in F$. Since x is a finite real vector (not extended-real-valued), $\|x\|_\infty < \infty$. Taking $M \geq \|x\|_\infty$, for coordinates where x_j originally has finite bounds, $\hat{l}_j(M) = l_j \leq x_j \leq u_j = \hat{u}_j(M)$ holds as is, and for coordinates that were originally unbounded, $|x_j| \leq M$ gives $-M \leq x_j \leq M$. Hence $x \in F_M$. $\square$

From the sign of the slope z^1 and the termination outcome, the three cases of the original problem can be distinguished exactly and exhaustively.

Proposition 6.5 (Classification into the three cases). *Suppose the combinatorial state transition sequence reaches the final state (B^*, η^*) beyond some $M^\dagger > 0$, where $(a')(b')(c)(d)$ terminates. Then the following hold.*

- (i) *The termination outcome is INFEASIBLE $\iff F = \emptyset$.*
- (ii) *The termination outcome is OPTIMAL and $z^1 < 0 \iff F \neq \emptyset$ and $\inf_{x \in F} c^\top x = -\infty$ (the original problem is feasible and unbounded).*
- (iii) *The termination outcome is OPTIMAL and $z^1 = 0 \iff F \neq \emptyset$ and $\inf_{x \in F} c^\top x$ is finite, in which case $\inf_{x \in F} c^\top x = z^0$.*

Proof. By Proposition 6.3 and the discussion following it, the termination outcome is the same for all $M \geq M^\dagger$, and either F_M is empty for all $M \geq M^\dagger$, or it is nonempty for all $M \geq M^\dagger$ with minimum value $z(M) = z^0 + z^1 M$. Moreover, $M_1 \leq M_2$ implies $F_{M_1} \subseteq F_{M_2}$ (monotonicity).

(i): ($\Leftarrow$) If $F = \emptyset$, then by Lemma 6.4, $F_M \subseteq F = \emptyset$ for every M . In particular, (3) is infeasible for $M \geq M^\dagger$, and the termination outcome is INFEASIBLE. ($\Rightarrow$) If the termination outcome is INFEASIBLE, then $F_{M^\dagger} = \emptyset$. By monotonicity, $F_M \subseteq F_{M^\dagger} = \emptyset$ also for $M < M^\dagger$. Hence, by Lemma 6.4, $F = \bigcup_M F_M = \emptyset$.

(ii)(iii): In what follows, by (i) we may assume $F \neq \emptyset$, so the termination outcome is OPTIMAL. By Lemma 6.4 and monotonicity, for any $x \in F$, setting $M_x := \max(M^\dagger, \|x\|_\infty)$, we have $x \in F_{M_x}$ (by the same argument as in the proof of Lemma 6.4), and therefore

$$c^\top x \geq \min_{F_{M_x}} c^\top x = z(M_x) = z^0 + z^1 M_x. \quad (*)$$

($z^1 < 0 \Rightarrow$ unbounded): Since $F_M \subseteq F$, $\inf_{x \in F} c^\top x \leq z(M) = z^0 + z^1 M \rightarrow -\infty$ ($M \rightarrow \infty$).

(Unbounded $\Rightarrow z^1 < 0$): We prove the contrapositive. Suppose $z^1 = 0$. If $F \neq \emptyset$ and the problem is unbounded, then for any $N > 0$ there exists $x^{(N)} \in F$ with $c^\top x^{(N)} < -N$. Applying (*) to $x = x^{(N)}$ gives $c^\top x^{(N)} \geq z^0 + z^1 M_{x^{(N)}} = z^0$ (independent of the value of $M_{x^{(N)}}$ since $z^1 = 0$). Hence $z^0 \leq c^\top x^{(N)} < -N$. Since N is arbitrary, this gives $z^0 = -\infty$, contradicting the fact that z^0 is defined as a finite real number by Lemma 6.1. Hence, if the problem is unbounded, then $z^1 \neq 0$. Combined with $z^1 \leq 0$ shown earlier, $z^1 < 0$.

From the above, when $F \neq \emptyset$, “ $z^1 < 0 \iff$ unbounded” holds, and therefore (by the dichotomy given by $z^1 \leq 0$) “ $z^1 = 0 \iff$ a finite optimal value exists” follows.

Finally, we show that $\inf_{x \in F} c^\top x = z^0$ in the case $z^1 = 0$. For any $x \in F$, using $z^1 = 0$ in (*) gives $c^\top x \geq z^0$. On the other hand, fixing an arbitrary $M \geq M^\dagger$, since the final state is primal feasible for $M \geq M^\dagger$, the basic solution $(x_B(M), x_N(M))$ at this M belongs to F_M , and $F_M \subseteq F$ by the ($\supseteq$) direction of the proof of Lemma 6.4. Moreover, $c_B^\top x_B(M) + c_N^\top x_N(M) = z(M) = z^0 + z^1 M = z^0$ (the last equality by $z^1 = 0$). Hence z^0 is a lower bound actually attained on F , and $z^0 = \min_{x \in F} c^\top x$ follows. $\square$

By Proposition 6.5, the distinction between an insufficient M and unboundedness, which was problematic in the artificial bounding method with a numerical M (Section 1), can be made exactly, without restarting, from the sign of the slope z^1 at termination alone.

In case (iii) of Proposition 6.5, the optimal *basis* (B^*, η^*) and the optimal value z^0 are obtained without fixing M , but extracting a concrete optimal solution vector requires the cleanup described below.

6.3 Cleanup of nonbasic variables remaining at artificial bounds

Consider the variables j that remain nonbasic in the final state (B^*, η^*) with $x_j^1 \neq 0$ (i.e., placed on the side that was infinite, so that $x_j(M) = \mp M$). Such a j is either a variable with a single infinite bound (whose bound on the finite side is exactly 0 by preprocessing) or a free variable.

By the standard simplex argument, rewriting $z(M) = c_B^\top x_B(M) + c_N^\top x_N(M)$ using $x_B(M) = A_B^{-1}(b - A_N x_N(M))$ gives

$$z(M) = c_B^\top A_B^{-1} b + \sum_{j \in N} \bar{c}_j x_j(M).$$

Extracting only the terms among the nonbasic values $x_j(M)$ that depend on M (those j placed at artificial bounds), the slope of $z(M) = z^0 + z^1 M$ is expressed as

$$z^1 = \sum_{\substack{j \in N, \eta(j)=L \\ l_j = -\infty}} (-\bar{c}_j) + \sum_{\substack{j \in N, \eta(j)=U \\ u_j = +\infty}} \bar{c}_j$$

(this expression equals $z^1 = c^\top x^1$ from Lemma 6.1 rewritten in terms of reduced costs). By dual feasibility, each term with $\eta(j) = L$ has $\bar{c}_j \geq 0$ and hence $-\bar{c}_j \leq 0$, and each term with $\eta(j) = U$ has $\bar{c}_j \leq 0$; thus z^1 is a *sum of nonpositive terms*, which also gives an alternative proof of $z^1 \leq 0$ shown earlier. In particular,

$$z^1 = 0 \iff \bar{c}_j = 0 \text{ for all nonbasic variables } j \text{ with } x_j^1 \neq 0$$

holds. This is nothing but the standard optimality condition that the reduced cost of a nonbasic variable placed in an unbounded direction must be 0.

However, when there exist nonbasic variables with $x_j^1 \neq 0$, the point $x^0 = (x_N^0, x_B^0)$ obtained by formally substituting $M = 0$ is in general not a feasible solution of the original problem. Although the nonbasic values x_N^0 lie on the true bounds by preprocessing, primal feasibility of the basic values $x_B(M) = x_B^0 + M x_B^1$ is guaranteed only for $M \geq M^\dagger$, and extrapolating down to $M = 0$ may violate the bounds. The following lemma resolves this issue by exploiting the property $\bar{c}_j = 0$.

Lemma 6.6 (Removal of nonbasic variables at artificial bounds by a primal ratio test). *Suppose $z^1 = 0$. Assume that the combinatorial state (B, η) is dual feasible for the original problem (1), and that each component of its basic solution $x(M)$ is an affine function of M satisfying the bounds of the original problem $l \leq x(M) \leq u$ for all sufficiently large M (the final state (B^*, η^*) satisfies this, since $F_M \subseteq F$ and dual feasibility does not depend on M). Let $j \in N$ be a nonbasic variable with $x_j^1 \neq 0$ (so $\bar{c}_j = 0$ by $z^1 = 0$). By preprocessing, the current value of j is $x_j(M) = x_j^1 M$, and if j has a single infinite bound, its bound on the finite side is exactly 0. If x_j is moved from $x_j^1 M$ toward 0 by a step $t \in [0, M]$ ($x_j \leftarrow x_j^1(M - t)$), the basic variables change as*

$$x_B \leftarrow x_B(M) + x_j^1 t \alpha_j, \quad \alpha_j := A_B^{-1} A_j.$$

For each row i , define the step at which $x_{B,i}$ reaches a genuinely finite bound of $\beta(i)$ as

$$t_i := \begin{cases} \frac{u_{\beta(i)} - x_{B,i}(M)}{x_j^1 \alpha_{ij}} & (x_j^1 \alpha_{ij} > 0, u_{\beta(i)} < +\infty) \\ \frac{x_{B,i}(M) - l_{\beta(i)}}{-x_j^1 \alpha_{ij}} & (x_j^1 \alpha_{ij} < 0, l_{\beta(i)} > -\infty) \end{cases}$$

(the other rows do not block the move; the artificial bounds $\pm M$ do not exist in the original problem and are therefore not taken into account), and let t^* be the minimal element of $\{M\} \cup \{t_i\}$ with respect to $\succ$ (all of these are affine functions of M ; ties are broken in favor of M , and among the t_i in favor of the smaller index). Then:

- (A) If $t^* = M$, set x_j to 0. That is, if j has a single infinite bound, place it on its finite side (status L or U); if it is a free variable, place it in status Z . The basis is not changed.
- (B) If $t^* = t_r < M$, perform a pivot with j as the entering variable and $\beta(r)$ as the leaving variable, and fix $\beta(r)$ at the finite bound it has reached (status U for an upper bound, status L for a lower bound).

Then the following hold.

- (i) The objective value does not change.
- (ii) Dual feasibility for the original problem (1) is preserved.
- (iii) The updated basic solution is also an affine function of M and satisfies $l \leq x(M) \leq u$ for all sufficiently large M .
- (iv) The number of nonbasic variables with $x_j^1 \neq 0$ decreases by exactly 1.

Proof. (i): The change in objective value due to the move is $c_j(-x_j^1 t) + c_B^\top (x_j^1 t \alpha_{\cdot j}) = -x_j^1 t (c_j - c_B^\top A_B^{-1} A_j) = -x_j^1 t \bar{c}_j = 0$.

(ii): In (A), the basis does not change, so the reduced costs are unchanged; only the status of j changes, and $\bar{c}_j = 0$ satisfies the condition for each of the statuses L, U, Z . In (B), by the tableau update formula, the reduced cost of every other nonbasic variable k is unchanged, $\bar{c}_k^{\text{new}} = \bar{c}_k - (\bar{c}_j / \alpha_{rj}) \alpha_{rk} = \bar{c}_k$, and the reduced cost of the leaving variable becomes $\bar{c}_{\beta(r)}^{\text{new}} = -\bar{c}_j / \alpha_{rj} = 0$, so the sign condition is satisfied regardless of which bound $\beta(r)$ is placed at.

(iii): Since t^* is an affine function of M , each updated component is also an affine function. By assumption, $x(M)$ satisfies the genuinely finite bounds for sufficiently large M , so each $t_i \geq 0$, and therefore $0 \leq t^* \leq M$. The nonbasic variable j stays between $x_j^1 M$ and 0, which is within the bounds of j . For basic variables i that may block the move, $t^* \leq t_i$ ensures that the slack to the bound remains ≥ 0 ; the other basic variables either move toward an infinite side or do not move. The variable $\beta(r)$ leaving in (B) lies exactly on a finite bound. The other nonbasic variables do not change. Here ≥ 0 coincides with numerical nonnegativity for sufficiently large M .

(iv): In (A), the nonbasic value of j becomes 0, so $x_j^1 = 0$. In (B), j enters the basis, and $\beta(r)$, which becomes nonbasic in its place, is placed at a finite bound ($x_{\beta(r)}^1 = 0$). The statuses of the other nonbasic variables do not change. $\square$

We apply Lemma 6.6 repeatedly until no nonbasic variables with $x_j^1 \neq 0$ remain. By (iv), if K denotes the number of such variables in the final state (B^*, η^*) , this terminates after exactly K applications. In the resulting basis $(\tilde{B}, \tilde{\eta})$ we have $x_N^1 = 0$, so the nonbasic values $x_N = x_N^0$ do not depend on M , and hence the basic values $x_B = A_{\tilde{B}}^{-1}(b - A_N x_N^0)$ are also constants independent of M . By (iii), this constant vector x^{**} satisfies $l \leq x^{**} \leq u$ for sufficiently large M .

M (and hence regardless of M), and is primal feasible for the original problem (1). Moreover, by (ii) it is dual feasible, so x^{**} is an optimal *basic* solution of the original problem, and by (i) $c^\top x^{**} = z(M) = z^0$ (consistent with Proposition 6.5(iii)).

Remark 6.7 (Use of status Z , and the case of a zero column). Status Z appears only in the initial placement of free variables with cost 0 ((4)) and when a free variable j is set to value 0 in (A) of Lemma 6.6. Since variables leaving the basis in the main iterations are placed at L or U , no new variable with status Z arises during the main iterations. Also, since $\beta(r)$ leaving in (B) of Lemma 6.6 is a variable that has reached a genuinely finite bound, no free variable leaves the basis during cleanup. Furthermore, when $\alpha_{.j} = A_B^{-1} A_j = 0$, no t_i is defined, so case (A) applies automatically, and it suffices to move j to its finite side (or to status Z) without changing x_B .

Remark 6.8 (Comparison with cleanup that pivots on an arbitrary row). Statements (i)(ii)(iv) of Lemma 6.6 also hold for a degenerate pivot that brings j with $x_j^1 \neq 0$ into the basis on an *arbitrary* row r with $\alpha_{rj} \neq 0$ and fixes $\beta(r)$ on its finite side. In this case, however, primal feasibility is lost because the value of $\beta(r)$ is forcibly moved to a bound, and the dual simplex method must be restarted after reaching $x_N^1 = 0$. The number of iterations of this restart cannot be estimated in advance. Lemma 6.6 preserves primal feasibility by choosing the leaving row via a ratio test, thereby making this restart unnecessary. Alternatively, an optimal solution can also be obtained by substituting a concrete numerical value of M (the smallest value that yields primal feasibility) into the basic solution $x(M)$ of the final state, but the resulting point is in general not a basic solution, and if some row has a small slope, M becomes extremely large and causes numerical errors. The procedure of Lemma 6.6 treats M symbolically throughout and finally yields a basic solution free of M , so this problem does not arise either.

Remark 6.9 (Relation to crossover procedures). *Crossover* [20, 21, 37] is a procedure that, starting from a solution in the relative interior of the optimal face obtained by an interior-point method, constructs an optimal basic solution by moving variables lying strictly between their bounds, one at a time, either to a bound or into the basis, while preserving the objective value; the stage that moves the primal variables is called the primal push, and the stage that moves the dual variables is called the dual push.

The operation of Lemma 6.6 has the same structure as the primal push in this crossover procedure. Indeed, the basic solution of the final state (B^*, η^*) is optimal for the original problem, but since the nonbasic variables with $x_j^1 \neq 0$ do not lie on bounds of the original problem, it is not a basic solution (vertex) of the original problem. Such a nonbasic variable j with $x_j^1 \neq 0$ is, from the viewpoint of the original problem, a nonbasic variable not placed at any finite bound (a so-called *superbasic* variable), and by $z^1 = 0$ its reduced cost is 0. The primal push moves such variables one at a time toward a bound, and if a basic variable reaches a bound first along the way, performs a pivot exchanging the two. The properties that the objective value and primal and dual feasibility are preserved because the reduced cost is 0, and that each operation decreases the number of such variables by exactly one (Lemma 6.6(i)–(iv)), are also shared. Megiddo [20] showed by an argument of this kind that an optimal basis can be obtained from a pair of primal and dual optimal solutions in at most n pivots. Lemma 6.6 applies this primal push to the symbolic setting in which the steps and the basic solution are affine functions of M , performing comparisons in the lexicographic order $\succ$. Note that, under the duality of Section 8, this corresponds on the dual side to driving out artificial variables that remained in the basis at value 0 while preserving optimality, i.e., to the dual push of crossover.

6.4 Non-return to infinite bounds and interpretation of the method

Once $x_N^1 = 0$ (the state in which all nonbasic variables are placed at genuinely finite bounds) is attained in the course of the iterations, infinite bounds never reappear as nonbasic variables in subsequent iterations. The crux of this property is the fact that $x_{B,r}$, being a finite real number, can never “violate” an infinite bound.

Proposition 6.10 (Non-return of nonbasic variables to infinite bounds). *Consider the combinatorial state transition sequence $(B^{(0)}, \eta^{(0)}) \rightarrow (B^{(1)}, \eta^{(1)}) \rightarrow \dots$ generated by the symbolic artificial bounding method (Steps 2(a')(b')(c)(d)). If for some $k_0 \geq 0$ all nonbasic variables of $(B^{(k_0)}, \eta^{(k_0)})$ have $x_j^1 = 0$ (i.e., are placed at genuinely finite bounds), then the same holds for all $k \geq k_0$.*

Proof. We proceed by induction on $k \geq k_0$. For $k = k_0$ the claim holds by assumption. Assume that for some $k \geq k_0$ all nonbasic variables of $(B^{(k)}, \eta^{(k)})$ have $x_j^1 = 0$, and consider one iteration $(B^{(k)}, \eta^{(k)}) \rightarrow (B^{(k+1)}, \eta^{(k+1)})$.

First, since $x_N^1 = 0$ in Lemma 6.1, we have $x_N(M) \equiv x_N^0$, and therefore $x_B^1 = -A_B^{-1} A_N x_N^1 = 0$, i.e., $x_B(M) \equiv x_B^0$ is a constant independent of M .

For each row i , if $\beta(i)$ has an infinite side (e.g., $l_{\beta(i)} = -\infty$), the corresponding violation $v_i^-(M) = \hat{l}_{\beta(i)}(M) - x_{B,i}(M) = (-M) - x_{B,i}^0$ has slope -1 and diverges to $-\infty$ as $M \rightarrow \infty$, so $v_i^- \succ 0$ never holds for $M \geq M^\dagger$, whatever $M^\dagger$ is. On the other hand, the violation corresponding to a genuinely finite side of $\beta(i)$, together with the fact that $x_{B,i}(M) = x_{B,i}^0$ is constant, is likewise a constant independent of M (the case $u_{\beta(i)} = +\infty$ is symmetric). Therefore, the violation $w_r(M) = w_r^1 M + w_r^0$ of the leaving row r is a constant (an ordinary real number) with slope $w_r^1 = 0$.

(i) **The leaving variable is fixed at a genuinely finite bound:** Since $w_r(M)$ is constant, r is necessarily selected because of a violation on a genuinely finite side of $\beta(r)$ (the violation on an infinite side diverges with slope -1 as seen above and cannot be $\succ 0$). By the pivot in Step 2(d), $\beta(r)$ is fixed at that genuinely finite bound and becomes nonbasic.

(ii) **Conclusion:** The entering variable q , which becomes basic in the pivot, is removed from the nonbasic set; by (i), $\beta(r)$, which becomes nonbasic in the pivot, is fixed at a genuinely finite bound; and the other nonbasic variables do not change status in this iteration. Therefore all nonbasic variables of $(B^{(k+1)}, \eta^{(k+1)})$ also have $x_j^1 = 0$. $\square$

Remark 6.11 ($x_N^1 = 0$ as an absorbing boundary and implementation consequences). From the proof of Proposition 6.10, once $x_N^1 = 0$ is reached, all violations become constants independent of M , and the tests in Step 2 (primal feasibility test, selection of the leaving row, ratio test) coincide exactly with those of the dual simplex method of Section 3.1 with respect to the true bounds l, u . That is, reaching $x_N^1 = 0$ functions as an absorbing boundary that makes the subsequent behavior coincide with the classical dual simplex method.

Remark 6.12 (On iterations after reaching $z^1 = 0$). The slope $z^1 = \sum_{j \in N} x_j^1 \bar{c}_j$ of $z(M)$ in the current state has only nonpositive terms by dual feasibility, and $-z^1$ corresponds to the dual infeasibility with respect to the original problem (1) (the sum of the absolute values of the reduced costs of the nonbasic variables placed on infinite sides). In the initial state, $z^1 = -\sum_{j \in S} |c_j|$. Since each iteration of the dual simplex method does not decrease the objective value for all sufficiently large M , (z^1, z^0) is lexicographically nondecreasing; in particular, z^1 is monotonically nondecreasing, and once it reaches 0 it remains 0 thereafter. In this sense, the symbolic artificial bounding method roughly corresponds to dual Phase 1 while $z^1 < 0$ and to Phase 2 after reaching $z^1 = 0$, and if $S = \emptyset$ it is already in the latter at the initial state. However, the precise division into phases is given not by the sign of z^1 but by Phase A and Phase B of Section 7 (Remark 7.8), and the sequence of iterates coincides exactly with the classical dual simplex method only after $x_N^1 = 0$ is reached (Remark 6.11).

Remark 6.13 (Interpretation as a guaranteed crash procedure). Since the dual feasibility condition is determined solely by the signs of the reduced costs and does not depend on M , the iterations of Step 2 are dual feasible for the original problem (1) as well, consistently from the start. What Step 2 actually resolves is merely the single point that some nonbasic variables are placed at the artificial values $\pm M$ ($x_N^1 \neq 0$). From this viewpoint, Step 2 (including the cleanup of Lemma 6.6) can be regarded as a *crash procedure* for obtaining a starting basis that

is dual feasible for the original problem and in which all nonbasic variables are placed at finite bounds. Whereas the classical crash procedures of Section 4 are heuristic and carry no feasibility guarantee, this procedure, under an anti-cycling rule (Section 6.5), *always* reaches $x_N^1 = 0$ after finitely many iterations (or detects unboundedness or infeasibility), and by Proposition 6.10 never moves back once it has been reached.

Remark 6.14 (Comparison with the cost shifting method). A technique widely used at the start of the dual simplex method is the *cost shifting method* [7]. It is a two-stage procedure that temporarily replaces the costs c_j of nonbasic variables violating the dual feasibility conditions by artificial values so as to make the basis *apparently* dual feasible, runs the dual simplex method under this artificial objective function, and then restores dual feasibility by additional pivots while returning the costs to their true values.

From the dual side, both the cost shifting method and our bounding by M are operations that temporarily relax the violated dual constraints $\bar{c}_j \leq 0$ or $\bar{c}_j \geq 0$. The difference is that the cost shifting method relaxes them *without penalty* by moving the right-hand side c_j of the dual constraint itself and later restores it, whereas our bounding relaxes them *with a penalty of coefficient M* , keeping the violation as an artificial variable (see Section 8 below). From the primal side, our approach does not artificialize the objective function and uses the true $c^\top x$ from beginning to end. Because the penalty incorporates the elimination of dual infeasibility into the objective function, advantages arise in the following two respects. First, whereas in the cost shifting method the number of additional pivots in the stage that undoes the shift cannot in general be characterized in advance, our cleanup is proved to finish in at most K pivots without changing the objective value and while preserving primal and dual feasibility (Lemma 6.6). Second, whereas in the cost shifting method the variables to be shifted and the shift amounts are chosen heuristically, in our approach the set S of variables requiring placement on an infinite side is uniquely determined by the data, and moreover an exact criterion for classifying the original problem into the three cases (Proposition 6.5) is obtained within the same framework.

6.5 Anti-cycling: extending Bland's rule to the lexicographic order

So far, in Steps 2(a')(b') as well as in the ratio test, we have only stated that ties among candidates are broken deterministically by a rule such as “give priority to the smaller index,” with the caveat that this alone cannot in general exclude cycling (repeatedly visiting the same basis). The standard ways to prevent cycling in the classical dual simplex method with bound constraints are to use Bland's rule (the smallest-index rule) [4] itself as the primary selection rule, or to use the *lexicographic rule* [2], which is compatible with any primary selection rule such as the Dantzig rule, the steepest-edge rule, or the Devex rule. The lexicographic rule in the dual simplex method resolves ties in the ratio test (Step 2(c)) by lexicographically comparing the reduced costs obtained when the costs c are perturbed by infinitesimals $\epsilon = (\epsilon_1, \epsilon_2, \dots)$, $\epsilon_1 \gg \epsilon_2 \gg \dots \gg 0$. Since the cause of cycling is dual degeneracy (ties with ratio 0 in the ratio test), perturbation is needed only on the ratio-test side, and ties in the selection of the leaving row (Step 2(b)) may be broken by any deterministic rule.

In our setting, this lexicographic rule does not interfere with the lexicographic order $\succ$ with respect to M . The ratios $\hat{c}_j/|\hat{\alpha}_j|$ compared in the ratio test do not depend on M , and neither do the perturbed reduced costs, so ties in the ratio test can be resolved solely by lexicographic comparison with respect to ϵ . On the other hand, the tests in which M appears (comparisons of violations in (a')(b')) are carried out solely with $\succ$ with respect to M , and ties in the leaving row may be broken by a rule such as the smallest index. That is, there is no need to prepare a lexicographic order concatenating M and ϵ .

Proposition 6.15 (Anti-cycling). *Suppose ties in the ratio test are resolved by the lexicographic rule based on the cost perturbation above, and all other ties are resolved by an arbitrary deterministic rule. Then the combinatorial iterations of the symbolic artificial bounding method never*

visit the same state (B, η) twice, and terminate after finitely many iterations.

Proof. We first handle variables with status Z . The costs of variables with status Z are not perturbed. When a variable with status Z is eligible to enter, it becomes basic with dual step length $\theta = 0$ (Remark 3.1), so the reduced costs are unchanged even after perturbation, and the reduced costs of variables with status Z remain exactly 0 in the perturbed problem as well. Each such pivot decreases the number of nonbasic variables with status Z by one, and no new variable with status Z arises during the main iterations (Remark 6.7). Hence pivots of this kind occur at most as many times as the initial number of variables with status Z . In what follows, we consider the other pivots.

In the perturbed problem, the reduced costs of all nonbasic variables except fixed variables and variables with status Z are lexicographically nonzero with respect to ϵ , and it is a standard fact of the classical lexicographic dual simplex method that this property is preserved throughout the iterations. Therefore the dual step length θ_q in each iteration is lexicographically positive with respect to ϵ . On the other hand, the violation of the leaving row satisfies $w_r \succ 0$, and the change in the objective value is $\theta_q w_r(M)$, so for sufficiently large M and sufficiently small ϵ the objective value strictly increases. Since there are finitely many states (B, η) and the objective value is determined by the state (and since the states are finite in number, the threshold on M can also be taken uniformly), the same state is never visited twice within an interval during which the number of variables with status Z is constant. Since the number of variables with status Z is monotonically nonincreasing, the same state is never visited twice overall, and the iterations terminate after finitely many steps. $\square$

That is, for anti-cycling it suffices to apply the classical lexicographic rule only on the ratio-test side, and the lexicographic order $\succ$ with respect to M can be used as is.

7 Decomposition into the slope problem and the intercept problem

From Section 6.1 onward, in the tests (a')(b') we have compared affine functions $f^1 M + f^0$ by the lexicographic order $\succ$ on the coefficient pairs (f^1, f^0) . In this section we show that in the main iterations there is essentially no situation in which the two components act simultaneously, and that the iterations split exactly into a “phase in which comparisons use only the slope f^1 ” and a “phase in which the slope stays fixed and comparisons use the intercept f^0 ”.

7.1 The slope problem

For each variable j , define the slopes of the bounds by

$$l_j^1 := \begin{cases} -1 & (l_j = -\infty) \\ 0 & (l_j > -\infty) \end{cases}, \quad u_j^1 := \begin{cases} 1 & (u_j = +\infty) \\ 0 & (u_j < +\infty) \end{cases}$$

and let l_j^0 be l_j if $l_j > -\infty$ and 0 if $l_j = -\infty$; define u_j^0 analogously (for slack variables, $l^1 = u^1 = l^0 = u^0 = 0$). Then (2) can be written as $\hat{l}_j(M) = l_j^0 + l_j^1 M$, $\hat{u}_j(M) = u_j^0 + u_j^1 M$. Hence problem (3), written with the zero-fixed slack variables s of Section 5.3, is

$$\min c^\top x \quad \text{s.t.} \quad Ax + s = b, \quad 0 \leq s \leq 0, \quad l^0 + l^1 M \leq x \leq u^0 + u^1 M \quad (6)$$

We set the right-hand side b and the constant parts l^0, u^0 to 0, keep only the coefficients l^1, u^1 of M as bounds, and consider the problem

$$\min c^\top x^1 \quad \text{s.t.} \quad Ax^1 + s = 0, \quad 0 \leq s \leq 0, \quad l^1 \leq x^1 \leq u^1 \quad (7)$$

which we call the *slope problem*. We denote the variables by x^1 because, as we will see in Lemma 7.1, its basic solution coincides exactly with the slope x^1 of Lemma 6.1. In both (6) and (7) we keep s as variables, since the combinatorial state (B, η) of the algorithm is defined on the indices of (x, s) and the basis B may contain slack variables. Moreover, $0 \leq s \leq 0$ are bounds, and in a basic solution that is not primal feasible a basic s_i may take a nonzero value.

The slope problem itself is not new: it coincides with the auxiliary problem that minimizes the sum of dual infeasibilities, known as Phase 1 of the dual simplex method [13, 14, 15, 16, 18]. Indeed, the auxiliary problem of Fourer [13] is the problem in which the bounds on the unbounded sides are set to $-1, +1$, respectively, and all other bounds and the right-hand side are set to 0; its bounds are precisely the definition of l^1, u^1 (Koberstein [16] removes the variables that are finite on both sides from the auxiliary problem, but these are fixed to $[0, 0]$ in the slope problem, so the two are equivalent). In these references, the auxiliary problem is derived directly as the minimization of the sum of dual infeasibilities and is treated as a Phase 1 technique separate from the artificial bounding method [16]. The derivation in this section connects the two by showing that this auxiliary problem arises as the coefficient of M (the slope) extracted from problem (3) of the artificial bounding method.

The slope problem is the recession cone of the original problem cut by the $\|\cdot\|_\infty$ ball, and it is always feasible at $x^1 = 0$. Indeed, let $F := \{x : Ax = b, l \leq x \leq u\}$ be the feasible region of the original problem (1) (after preprocessing), and set

$$K_j := \begin{cases} \{0\} & \text{(finite on both sides)} \\ [0, +\infty) & \text{(only the lower bound finite)} \\ (-\infty, 0] & \text{(only the upper bound finite)} \\ \mathbb{R} & \text{(free variable)} \end{cases}, \quad C := \{d : Ad = 0, d_j \in K_j \ (j = 1, \dots, n)\}$$

C is the cone obtained from F by replacing the right-hand side and the finite bounds by 0, and when $F \neq \emptyset$ it coincides with the recession cone $\text{rec}(F)$ of F . Since $[l_j^1, u_j^1] = K_j \cap [-1, 1]$ and, because $s = 0$, $Ax^1 + s = 0$ is equivalent to $Ax^1 = 0$, the feasible region of (7) is $C \cap \{d : \|d\|_\infty \leq 1\}$. Writing $x = Mx^1 + x^0$ in (3), dividing by M , and letting $M \rightarrow \infty$ yields (7); thus (7) is the problem that extracts the “slope” of (3) as $M \rightarrow \infty$. It is well known [9] that, when $F \neq \emptyset$, “the original problem is unbounded $\iff$ there exists $d \in \text{rec}(F)$ with $c^\top d < 0$ ”, which is consistent with the distinction $z^1 < 0 / z^1 = 0$ in Proposition 6.5(ii)(iii).

Lemma 7.1 (Relation between the basic solutions of (6) and (7)). *Fix a combinatorial state (B, η) , and write the basic solution of (6) for this state as $x(M) = x^0 + Mx^1$ (Lemma 6.1). Then the slope x^1 coincides with the basic solution of (7) for the same state, and the intercept is $x^0 = x(0)$ (both are uniquely determined by (B, η)).*

Proof. Since (6) and (7) share the constraint matrix $[A \mid I]$, the same (B, η) determines a basic solution in both. The bounds are $l^0 + l^1 M$, $u^0 + u^1 M$ and l^1, u^1 , respectively.

Let $\xi = (\xi_N, \xi_B)$ be the basic solution of (7).

Nonbasic part: if $\eta(j) = L$, then $x_j(M) = l_j^0 + l_j^1 M$ and $\xi_j = l_j^1$. If $\eta(j) = U$, then $x_j(M) = u_j^0 + u_j^1 M$ and $\xi_j = u_j^1$. If $\eta(j) = Z$, then $x_j(M) = \xi_j = 0$. Therefore $x_N(M) = x_N^0 + M\xi_N$, and $\xi_N = x_N^1$. Indeed, $l_j^1 = -1$ holds only when $l_j = -\infty$, and $u_j^1 = 1$ only when $u_j = +\infty$, so ξ_j coincides with the definition of x_j^1 in Lemma 6.1.

Basic part: the right-hand side of the equality constraints of (6) is b , and that of the equality constraints $Ax^1 + s = 0$ of (7) is 0, so

$$x_B(M) = A_B^{-1}(b - A_N x_N^0) + M A_B^{-1}(0 - A_N \xi_N) = x_B^0 + M \xi_B$$

and $\xi_B = -A_B^{-1} A_N x_N^1 = x_B^1$. Hence $\xi = x^1$, and setting $M = 0$ gives $x^0 = x(0)$. $\square$

That is, once (B, η) is fixed, the basic solution is a linear function of (right-hand side, bounds); since the data of (6) decompose as $(b, l^0, u^0) + M(0, l^1, u^1)$, the basic solution of problem (7), whose data are the coefficient part $(0, l^1, u^1)$ of M , is the slope of $x(M)$. Consequently:

- The slope $v_i^{\pm,1}$ of the violation $v_i^\pm = v_i^{\pm,1}M + v_i^{\pm,0}$ in (3) equals the violation of row i in (7).
- The reduced costs and the ratios $\hat{c}_j/|\hat{\alpha}_j|$ are determined only by the cost c and the basis B , and are therefore common to both problems.
- The slope $z^1 = c^\top x^1$ of $z(M) = z^0 + z^1M$ equals the objective value of the basic solution of (7).

7.2 Two-phase decomposition

Proposition 7.2 (Two-phase decomposition). *Consider the combinatorial state transitions of the symbolic artificial bounding method, and distinguish cases according to which of*

$$\text{Phase A: } \exists i, v_i^{-,1} > 0 \text{ or } v_i^{+,1} > 0, \quad \text{Phase B: } \forall i, v_i^{-,1} \leq 0 \text{ and } v_i^{+,1} \leq 0$$

the state (B, η) is in.

- (i) *In Phase A, select the leaving row using only the slopes w_i^1 (ties among rows with the largest slope are broken by an arbitrary deterministic rule), and let $w_r = w_r^1M + w_r^0$ be the violation of the selected leaving row r (that is, w_r^1 is whichever of $v_r^{-,1}, v_r^{+,1}$ is positive; since $v_r^{-,1} + v_r^{+,1} = l_{\beta(r)}^1 - u_{\beta(r)}^1 \leq 0$, the two are never positive simultaneously). Then, without using the second component of $\succ$ at all, each iteration is valid as an iteration of the dual simplex method for (3) with sufficiently large M . This iteration is exactly the dual simplex method for (7), and $\text{Eligible} = \emptyset$ never occurs. Phase A terminates after finitely many iterations, and z^1 at termination equals the optimal value of (7).*
- (ii) *Once the state enters Phase B, in all subsequent iterations the slopes of the values and bounds of all variables, and hence all $v_i^{\pm,1}$ and z^1 , remain unchanged, and the state never returns to Phase A.*

Proof. (i): The row r selected in Phase A satisfies $w_r^1 > 0$, so $w_r \succ 0$, that is, $r \in V_\infty$. In the dual simplex method, choosing any violated row as the leaving row preserves dual feasibility and does not decrease the dual objective; hence choosing this r in (b') instead of the maximal element with respect to $\succ$ is still valid as an iteration of the dual simplex method for (3) with sufficiently large M . The ratio test does not depend on M . Since all of the above tests are carried out using only quantities of (7), this iteration is precisely the dual simplex method for (7). Since (7) is feasible at $x^1 = 0$, $\text{Eligible} = \emptyset$, which would certify its infeasibility, does not occur. Anti-cycling follows from the same argument as in Section 6.5 (the objective value z^1 of (7) increases strictly lexicographically with respect to ϵ), and at termination primal feasibility of (7), that is, $v_i^{\pm,1} \leq 0$ for all i , holds.

(ii): The row selected in Phase B satisfies $w_r^1 = 0$, $w_r^0 > 0$. The primal step length in the pivot is $w_r/|\hat{\alpha}_q|$, whose slope is 0, so neither the slopes x_B^1 of the basic variables nor the slope x_q^1 of the entering variable q change. The leaving variable $\beta(r)$ is fixed at the bound on the violated side, but since $w_r^1 = 0$, the slope of that bound equals the slope of the value of $\beta(r)$. Therefore the slopes of the values of all variables are unchanged, and the slopes of the bounds are of course unchanged, so each $v_i^{\pm,1}$ and $z^1 = c^\top(\text{slope vector})$ are also unchanged. For the entering variable q , its slope x_q^1 equals l_q^1 or u_q^1 (0 if in status Z), so the slopes $l_q^1 - x_q^1$, $x_q^1 - u_q^1$ of the new row are both ≤ 0 . Hence Phase B is preserved. $\square$

Corollary 7.3 (Classification obtained at the end of Phase A). *For z^1 at the end of Phase A (the optimal value of the slope problem (7)), the following hold.*

- (i) If $z^1 < 0$, then the original problem (1) is unbounded or infeasible, and does not have a finite optimal value.
- (ii) If $z^1 = 0$, then the original problem (1) has a finite optimal value or is infeasible, and is not unbounded.

Proof. By Proposition 7.2(i), z^1 is the optimal value of the slope problem, whose feasible region is $C \cap \{d : \|d\|_\infty \leq 1\}$ as seen in Section 7.1. Since C is a cone, $z^1 < 0$ is equivalent to the existence of $d \in C$ with $c^\top d < 0$. If $F = \emptyset$ the claim is trivial, so assume $F \neq \emptyset$. Then $C = \text{rec}(F)$. (i): Taking $d \in \text{rec}(F)$ with $c^\top d < 0$ and $x \in F$, we have $x + td \in F$ and $c^\top(x + td) \rightarrow -\infty$ ($t \rightarrow \infty$), so the problem is unbounded. (ii): $c^\top d \geq 0$ for all $d \in \text{rec}(F)$. A linear function on a nonempty polyhedron that is nonnegative on the recession cone is bounded below and attains its minimum, so the problem has a finite optimal value. $\square$

That is, Phase A determines whether the original problem can have a finite optimal value (from the dual side, whether the original dual problem is feasible; Section 8.2). On the other hand, whether the original problem is feasible is not determined by Phase A alone. The original problem may be infeasible even when $z^1 = 0$, and likewise when $z^1 < 0$ (the example in Remark 7.8). This distinction is made in Phase B (Proposition 6.5(i)), and since z^1 does not change throughout Phase B (Proposition 7.2(ii)), the classification of Phase A does not contradict the final classification. In particular, if the original problem is known separately to be feasible, unboundedness is determined at the end of Phase A, and Phase B need not be executed.

7.3 The intercept problem

By Proposition 7.2(ii), the slopes of the values of all variables are constant in Phase B. That is, the slope $x^1 = (x_N^1, x_B^1)$ is constant throughout Phase B. At the end of Phase A, x^1 is a feasible solution of (7), so $Ax^1 = 0$ and $l^1 \leq x^1 \leq u^1$. Fixing this x^1 , writing $x = x^0 + Mx^1$, and rewriting the constraints of (3) for sufficiently large M as conditions on x^0 , we obtain the problem

$$\begin{aligned} \min \quad & c^\top x^0 \quad \text{s.t.} \quad Ax^0 + s = b, \quad 0 \leq s \leq 0, \quad l^B \leq x^0 \leq u^B \\ & l_j^B := \begin{cases} -\infty & (x_j^1 > l_j^1) \\ l_j & (x_j^1 = l_j^1 = 0) \\ 0 & (x_j^1 = l_j^1 = -1) \end{cases}, \quad u_j^B := \begin{cases} +\infty & (x_j^1 < u_j^1) \\ u_j & (x_j^1 = u_j^1 = 0) \\ 0 & (x_j^1 = u_j^1 = 1) \end{cases} \end{aligned} \quad (8)$$

which we call the *intercept problem*. Indeed, if $x_j^1 > l_j^1$, then $x_j = x_j^0 + Mx_j^1$ automatically satisfies the lower bound $\hat{l}_j(M) = l_j^0 + l_j^1 M$ for sufficiently large M , so the lower bound disappears; if $x_j^1 = l_j^1 = -1$, then $-M \leq x_j^0 - M$ leaves $x_j^0 \geq 0$; and if $x_j^1 = l_j^1 = 0$, then $l_j \leq x_j^0$ remains as is (similarly for upper bounds). The objective decomposes as $c^\top x = c^\top x^0 + Mz^1$, and $z^1 = c^\top x^1$ is constant throughout Phase B. (8) is the problem whose feasible region is the set of points that remain feasible for sufficiently large M when the original problem is shifted by M in the direction x^1 .

Proposition 7.4 (Coincidence of Phase B with the classical dual simplex method). *The iterations of the symbolic artificial bounding method in Phase B ((a')(b')(c)(d) of Step 2) coincide, iteration by iteration, with the iterations of the classical dual simplex method of Section 3.1 applied to problem (8), under the same leaving-row selection rule and tie-breaking. In particular, if $x^1 = 0$, then (8) is the original problem (1) itself, and Phase B coincides with the classical dual simplex method for the original problem.*

Proof. We show that each test of the two iterations sharing the combinatorial state (B, η) gives the same result.

- **Positions of nonbasic variables:** the slope x_j^1 of a nonbasic variable j equals l_j^1 or u_j^1 (0 if in status Z), so j lies on a finite bound $x_j^0 \in \{l_j^B, u_j^B\}$ of (8). A variable in status Z satisfies $l_j^1 = -1 < 0 = x_j^1 < 1 = u_j^1$, so it is a free variable in (8) as well, and is placed at the value 0. The basic part is x_B^0 (Lemma 7.1).
- **Dual feasibility and the ratio test:** the reduced costs, $\hat{c}_j$, and the ratios $\hat{c}_j/|\hat{a}_j|$ are determined only by the cost c and the basis B , and the sign conditions for the statuses L, U, Z are also common, so they coincide in both.
- **Identification of violated rows and selection of the leaving row:** in Phase B, $v_i^{\pm,1} \leq 0$ for all i . A side with $v_i^{\pm,1} < 0$ is equivalent to the slope of $\beta(i)$ being strictly away from the slope of the bound on that side, that is, to the bound on that side being infinite in (8), and is not regarded as violated in either. On a side with $v_i^{\pm,1} = 0$, the intercept $v_i^{\pm,0}$ itself equals the violation in (8). Therefore V_∞ coincides with the set of violated rows of (8), and comparison by $\succ$ reduces to comparison of intercepts, so the selection of the leaving row also coincides.
- **Infeasibility test:** Eligible = $\emptyset$ is the same condition as the infeasibility test of the classical dual simplex method for (8).

Finally, if $x^1 = 0$, then $x_j^1 > l_j^1$ is equivalent to $l_j^1 = -1$, that is, $l_j = -\infty$, and $x_j^1 = l_j^1 = 0$ is equivalent to $l_j > -\infty$, so $l^B = l$. Similarly $u^B = u$, and (8) coincides with (1). $\square$

Remark 7.5 (Relation to Remark 6.11). If $x_N^1 = 0$, then $x_B^1 = 0$ and hence $x^1 = 0$, so Remark 6.11 corresponds to the case $x^1 = 0$ of Proposition 7.4. On the other hand, Phase B is entered with $x^1 \neq 0$ when $z^1 < 0$, or when $z^1 = 0$ but nonbasic variables with reduced cost 0 remain at $\pm M$. In this case (8) is the problem obtained by removing some of the bounds of the original problem and adding bounds 0 originating from the artificial bounds, and the cleanup of Lemma 6.6 can be interpreted as the operation of removing x^1 and returning to the original problem. Note that since comparisons in Phase B are only between intercepts, Proposition 7.4 also holds for the symbolic artificial bounding method with the original lexicographic comparison.

7.4 Overall picture: a chain of problems solved while passing on the basis

By Propositions 7.2 and 7.4, the symbolic artificial bounding method, and hence the slope–intercept two-stage dual simplex method (Section 9), can be understood as the following chain.

Stage	Problem solved	Method	Starting basis
Phase A	Slope problem (7)	Dual simplex method	All-slack basis (dual feasible without computation)
Phase B	Intercept problem (8)	Dual simplex method	Final basis of Phase A
Cleanup	Original problem (1)	primal push (Lemma 6.6)	Final basis of Phase B

If Phase B stops with infeasibility, the original problem is infeasible; if it stops at an optimum and $z^1 < 0$, the original problem is unbounded; in either case the method terminates there. When $z^1 = 0$, if $x^1 = 0$ the intercept problem is the original problem itself, so no cleanup is needed (Proposition 7.4), and if $x^1 \neq 0$ the cleanup removes x^1 . That is, the slope–intercept two-stage dual simplex method is equivalent to the procedure “solve two LPs in sequence by the dual simplex method and, if necessary, move to a basis of the original problem by crossover”.

Passing on the basis is valid because the three problems share the constraint matrix A and the cost c , so the sign conditions on the reduced costs (dual feasibility) carry over unchanged, and because the nonbasic variables of the final basis of Phase A lie on finite bounds of the intercept problem (proof of Proposition 7.4). The cleanup preserves the objective value and primal and dual feasibility (Lemma 6.6).

Note, however, the following two points. First, the intercept problem is not determined by the data alone; it depends on the optimal basic solution x^1 obtained in Phase A. If the optimal solution of the slope problem is not unique, the intercept problem differs depending on which optimal solution is reached; thus this is not a procedure that solves fixed LPs in sequence, but a chain in which the solution of the preceding stage determines the problem of the next stage. Second, the equivalence for Phase A is exact for the variant that compares only slopes (Proposition 7.2(i)); in the version with the original lexicographic comparison, discriminating between rows with tied slopes by their intercepts corresponds to a particular choice of tie-breaking in the dual simplex method for the slope problem. Conversely, if ties in the slopes are broken arbitrarily in Phase A, exact agreement with the iteration sequence for a sufficiently large numerical M (Proposition 6.3) is in general lost, but dual feasibility, anti-cycling, and the classification into the three cases still hold.

Remark 7.6 (Phase A may be replaced by the primal simplex method). What is passed from Phase A to Phase B in the chain is only the optimal basis of the slope problem (7), that is, a state (B, η) that may include slack variables, which is primal feasible for (7) (the Phase B condition $v_i^{\pm,1} \leq 0$) and dual feasible for (3). Since Proposition 7.2(ii), Proposition 7.4, and Corollary 7.3 use only this property, it does not matter by which method that basis is obtained, and the primal simplex method may be used instead of the dual simplex method. Indeed, under the slack basis B_0 , if each nonbasic variable is placed on the side with slope 0 (on the finite side if one exists, and free variables in status Z), the basic solution is $x^1 = 0$, $s = 0$, and since the right-hand side of (7) is 0, this is a primal feasible basis. Hence the primal simplex method can start without a Phase 1 and reaches an optimal basis after finitely many iterations. In that basis, the reduced costs of the nonbasic variables with $l_j^1 < u_j^1$ satisfy the sign conditions according to the statuses L, U, Z . No sign condition is imposed on the variables finite on both sides, which are fixed to $[0, 0]$ in (7), but since their slope is 0, reassigning $\eta(j)$ according to the sign of the reduced cost (Remark 5.2; the same operation as (1) in Section 7.5) makes the basis dual feasible for (3) without changing x^1 . In this case, however, the iterations of Phase A are no longer iterations of the dual simplex method for (3), and the view of the slope–intercept two-stage dual simplex method as a single symbolic dual simplex method for (3) (the symbolic artificial bounding method) holds only for Phase B and the cleanup. Moreover, since the optimal solution of the slope problem is not necessarily unique, the x^1 reached, and hence the intercept problem, generally differ from those obtained with the dual simplex method. Note also that at the starting point $x^1 = 0$ all basic variables are 0 and primal degeneracy is severe, so which approach is more efficient depends on the problem; we do not evaluate this in this paper.

Remark 7.7 (The separation does not essentially change the variable selection rules). The fact that the problem separates into the slope problem and the intercept problem does not yield new variable selection rules specific to each problem. Both the steepest-edge weights $\gamma_i = \sum_{j \in N} \alpha_{ij}^2$ and the approximate weights of the Devex rule are determined only by the constraint matrix $[A \mid I]$ and the basis B (and the pivot history), and do not depend on the right-hand side or the bounds. Since the three problems (6), (7), and (8) share the constraint matrix, the weights at the same basis coincide for the three problems. Therefore “the steepest-edge rule for the slope problem” is nothing but the normalization of the slope violations w_i^1 by the steepest-edge weights of the original problem, and “the steepest-edge rule for the intercept problem” is the normalization of the violations of the intercept problem by the same weights. The same applies to the ratios $\hat{c}_j/|\hat{\alpha}_j|$ for the entering variable, since they are also determined only by c and B . There is the implementation advantage that the weights can be carried over across phases as they are and no recomputation is required when switching phases, but one cannot expect the separation to improve the quality of selection (there is room to reset the Devex reference framework at the start of Phase B, but its effect would need to be verified empirically).

Remark 7.8 (On the boundary between the phases). The boundary between Phases A and B in Proposition 7.2 differs both from “while M remains ($x^1 \neq 0$)” and from “while $z^1 < 0$ ”.

- (a) Even while M remains, there can be rows with $w_i^1 = 0$, $w_i^0 > 0$. For example, in

$$\min x_1 \quad \text{s.t.} \quad x_2 = 5, \quad x_1 \text{ free}, \quad 0 \leq x_2 \leq 1$$

$x_1 = -M$ ($x_1^1 = -1$) in the initial state, but the violation of row 1 is $v_1^+ = 5$ (slope 0). If (a') is tested using only the first component, then $V_\infty = \emptyset$ and $z^1 = -1 < 0$, and the infeasible problem is misclassified as “unbounded”.

- (b) Even if $z^1 = 0$ occurs in the middle of Phase A, Phase A may continue due to dual degenerate pivots. Conversely, since z^1 is fixed in Phase B, if $z^1 < 0$ at the end of Phase A, Phase B proceeds with $z^1 < 0$, and whether the problem is infeasible or unbounded is determined in Phase B. The substance of the dual Phase 1 is Phase A, that is, the process of solving (7).

In the cleanup of Lemma 6.6, on the other hand, intercepts are needed when the slope of t_i coincides with that of M or when comparing t_i 's with equal slopes, and the lexicographic comparison using both components is indispensable.

7.5 Comparison with existing two-phase dual simplex methods

Existing method. The slope problem solved in Phase A coincides with the auxiliary problem known as Phase 1 of the dual simplex method [13, 14, 15, 16, 18] (Section 7.1). The existing two-phase dual simplex method using this auxiliary problem (e.g., Algorithm 8 of [16]) can be described as follows.

- (1) Variables finite on both sides are placed at one of their bounds according to the sign of the reduced cost (dual feasibility correction). As a result, the dual infeasibilities handled in Phase 1 are only those arising from variables with an unbounded side.
- (2) Construct the auxiliary problem in which the right-hand side is set to 0 and the bounds of variables with an unbounded side are replaced by $[0, 1]$ for variables with only a finite lower bound, $[-1, 0]$ for variables with only a finite upper bound, and $[-1, 1]$ for free variables (variables finite on both sides are removed). Since all variables of this problem are bounded, it can be solved directly, starting from any basis, by an ordinary dual simplex method (Phase 2 code) equipped with the steepest-edge rule and the BFRT. Its optimal value z_0 , with the sign reversed, is the minimum of the sum of infeasibilities of the original dual problem.
- (3) If $z_0 < 0$, the original problem is dual infeasible, and the method stops. In this case it is not distinguished whether the original problem is unbounded or infeasible, and making this distinction requires a separate procedure such as the primal simplex method. For example, when the dual simplex method of HiGHS (version 1.15.1) stops in this state, under the default settings it runs the primal simplex method anew to distinguish the two [26].
- (4) If $z_0 = 0$, the obtained basis is dual feasible for the original problem. Restore the original bounds and right-hand side. The nonbasic variables that were at the bounds ± 1 in the auxiliary problem have reduced cost 0, so they may be placed on the finite side (for free variables, in the nonbasic status with value 0). Then apply the dual simplex method (Phase 2) to the original problem.

That is, the existing method is a two-phase method consisting of Phase 1 with the auxiliary problem and Phase 2 for the original problem, and M appears nowhere. The artificial bounding method is treated as a Phase 1 technique separate from this [16].

Differences from Algorithm 1. Phase A solves the same problem as the Phase 1 in (2). The differences between the two appear as follows, depending on z^1 and x^1 at the end of Phase A.

- (a) If $z^1 = 0$ and $x^1 = 0$, the intercept problem is the original problem itself (Proposition 7.4), and the two coincide.
- (b) If $z^1 = 0$ and $x^1 \neq 0$, each term of $z^1 = c^\top x^1 = \sum_{j \in N} \bar{c}_j x_j^1$ is nonpositive by dual feasibility, so all nonbasic variables with $x_j^1 \neq 0$ (variables on artificial bounds) satisfy $\bar{c}_j = 0$. The existing method repositions these on the finite side in (4) to make $x^1 = 0$, and proceeds to Phase 2 for the original problem. Algorithm 1 leaves them on the artificial bounds, solves the intercept problem, and finally removes x^1 by the cleanup of Lemma 6.6. The basis and the basic solution x_B^0 at the start of Phase B are the same in both (owing to preprocessing, the intercept values l_j^0, u_j^0 of the variables on artificial bounds and the finite bounds they are repositioned to are all 0), but the problems solved in Phase B differ in their bounds on the support of x^1 , and the iterations and the final optimal basis generally differ. The optimal value is z^0 in both cases.
- (c) If $z^1 < 0$, the existing method stops in (3) without distinguishing unboundedness from infeasibility. Algorithm 1 solves the intercept problem by the dual simplex method directly from the basis of Phase A, and makes this distinction using the dual simplex method alone (Proposition 6.5). If only the distinction were needed, one could instead apply the dual simplex method from the basis of Phase A to the problem with cost 0 (for which every basis is dual feasible) and test primal feasibility. Algorithm 1 requires no such case distinction on the sign of z^1 : with the same Phase B procedure as in (a) and (b), it performs classification and solution simultaneously. To the best of our knowledge, no reference describes this treatment.

A unified view. The two methods in (b) can be understood in a unified way within the framework of (3). In (3) with sufficiently large M , a nonbasic variable with reduced cost 0 is dual feasible at either its upper or lower bound, and how it is placed is a degree of freedom inherent in the dual simplex method (an operation of the same kind as a flip in the BFRT or the dual feasibility correction in (1)). If the variables on artificial bounds are left as they are and we proceed to Phase B, then by Proposition 7.4 Phase B is the dual simplex method for the intercept problem, and cleanup is required. This is Algorithm 1. If at the boundary between the phases these are flipped to the opposite finite bound (for free variables, placed in status Z), then $x^1 = 0$, and by Proposition 6.10 and Remark 6.11 Phase B becomes the classical dual simplex method for the original problem, and no cleanup is required. This is the existing method. That is, when $z^1 = 0$, the two are executions of the same symbolic artificial bounding method that differ only in how the dual degenerate nonbasic variables are placed, and the existing method is derived by adding a single flip to the framework of this paper. The derivation in the reverse direction is asymmetric: from within the existing framework, which has no M , neither the intercept problem nor the cleanup arises naturally. For them to arise, a parametrization in M is needed, namely that the optimal solution takes the form $x^0 + Mx^1$. Moreover, when $z^1 < 0$, variables with $\bar{c}_j \neq 0$ remain on the artificial bounds, and flipping them would destroy dual feasibility, so the repositioning cannot be applied.

Scope of novelty. From the above, the slope–intercept two-stage dual simplex method solves the same problem as the existing Phase 1 in Phase A, but differs from the existing method as an algorithm in that it solves the intercept problem in Phase B (the contributions of this paper are summarized in Section 2). In case (b), which of the existing repositioning and the path of this paper is more efficient is not evaluated in this paper, and no theoretical superiority of either is established.

8 Interpretation from the dual problem

8.1 The big- M method for the dual problem

Viewed from the dual side, the construction so far is nothing but the application of the classical big- M method to the dual problem of the original problem, with M treated symbolically. In this section we summarize this correspondence.

Let $J^- := \{j : l_j = -\infty\}$ and $J^+ := \{j : u_j = +\infty\}$ be the sets of indices with an unbounded side. Restating the bounded problem (3),

$$\min c^\top x \quad \text{s.t.} \quad Ax = b, \quad \hat{l} \leq x \leq \hat{u} \quad (3)$$

where, by (2), $\hat{l}_j = -M$ ($j \in J^-$), $\hat{u}_j = M$ ($j \in J^+$), and otherwise $\hat{l}_j = l_j$, $\hat{u}_j = u_j$. With dual variables $\kappa, \nu \in \mathbb{R}^n$ for the lower- and upper-bound constraints, the dual problem of this problem is given by

$$\begin{aligned} \max \quad & b^\top y + \sum_{j \notin J^-} l_j \kappa_j - \sum_{j \notin J^+} u_j \nu_j - M \left(\sum_{j \in J^-} \kappa_j + \sum_{j \in J^+} \nu_j \right) \\ \text{s.t.} \quad & A^\top y + \kappa - \nu = c, \quad \kappa, \nu \geq 0 \end{aligned} \quad (9)$$

(the zero-fixed slacks s do not change the primal problem, so they may be omitted here). On the other hand, since the original problem (1) has no bound constraints on the unbounded sides, its dual problem is (9) with

$$\kappa_j = 0 \quad (j \in J^-), \quad \nu_j = 0 \quad (j \in J^+)$$

imposed (so that the penalty term vanishes), and it has the dual constraints $\bar{c}_j \leq 0$ ($j \in J^-$), $\bar{c}_j \geq 0$ ($j \in J^+$) on $\bar{c}_j := c_j - A_j^\top y$ ($\bar{c}_j = 0$ for free variables). That is, κ_j ($j \in J^-$) and ν_j ($j \in J^+$) are *artificial variables* that absorb the violations of these dual constraints, and (9) is exactly the big- M problem obtained by introducing artificial variables into the original dual problem and imposing a penalty with coefficient M . In other words, replacing the unbounded bounds of the primal problem by $\mp M$ is the operation dual to relaxing the constraints of the dual problem by a penalty with coefficient M . This is the primal-dual counterpart of the well-known fact that the dual of the big- M method applied to a primal problem is the problem in which the dual variables are bounded by M .

Furthermore, the dual simplex method for the primal problem is the primal simplex method for the dual problem carried out on the tableau of the primal problem [3, 9]. Indeed, for a combinatorial state (B, η) , if we set the basic dual solution determined by complementarity as

$$y^\top = c_B^\top A_B^{-1}, \quad \kappa_j = \bar{c}_j \quad (j \in N, \eta(j) = L), \quad \nu_j = -\bar{c}_j \quad (j \in N, \eta(j) = U)$$

(with the other components 0), then the equality constraints $A^\top y + \kappa - \nu = c$ are automatically satisfied, and $\kappa, \nu \geq 0$ coincides exactly with the dual feasibility condition of Section 3.1. The objective value of (9) is then $b^\top y + \sum_{j \in N} \bar{c}_j x_j(M)$, which equals $z(M) = z^0 + z^1 M$ seen in Section 6.1, and its penalty part is

$$-z^1 = \sum_{\substack{j \in N \cap J^- \\ \eta(j)=L}} \kappa_j + \sum_{\substack{j \in N \cap J^+ \\ \eta(j)=U}} \nu_j,$$

that is, the sum of the values of the artificial variables (the infeasibility with respect to the original dual problem). Therefore our algorithm coincides with the big- M method with M treated symbolically, carried out by the primal simplex method on the dual problem (9). The components correspond as follows.

- **Trivial initial basis** (Section 5.3): at $y = 0$ we have $\bar{c} = c$, the artificial variables are positive exactly for $j \in S$, and their values are $|c_j|$. This is the same structure as in the big- M method, where a trivial initial basis is obtained by letting artificial variables absorb constraint violations. However, artificial variables are needed not for all rows but only for the violated dual constraints $j \in S$. Also, there is no need to add columns for the artificial variables explicitly; placing the nonbasic variables of the primal problem at $\mp M$ represents them implicitly.
- **Nonbasic variables on unbounded bounds**: a nonbasic variable j with $x_j^1 \neq 0$ corresponds to the artificial variable κ_j or ν_j (with value $|\bar{c}_j|$) remaining in the basis of the dual problem.
- **Lexicographic comparison** $\succ$ (Section 6.1): the lexicographic comparison of coefficient pairs (z^1, z^0) corresponds to lexicographic optimization [10] that gives priority to minimizing the sum of artificial variables (dual infeasibility) over the original objective, and can be regarded as the big- M method treating M as a non-Archimedean infinity [11] applied to the dual problem. Fixing M as a number gives the artificial bounding method [18, 19]. The correspondence between the two components of $\succ$ and the dual Phase 1 and Phase 2 is described in Section 8.2.
- **Classification into the three cases** (Proposition 6.5): $z^1 < 0$ at termination corresponds to artificial variables remaining positive for all sufficiently large M , that is, to the original dual problem being infeasible, and together with $F \neq \emptyset$ this implies unboundedness of the original problem. Termination by $\text{Eligible} = \emptyset$ corresponds to the case where (9) is unbounded. These are the classical tests of the big- M method applied to the dual problem.
- **Existence of $M^\dagger$** (Lemma 6.4): the multipliers of the penalized dual constraints are the primal variables x_j themselves, so the argument of Lemma 6.4 that $x \in F_M$ if $M \geq \|x\|_\infty$ corresponds to the exact-penalty threshold argument that an L_1 penalty becomes exact once the penalty coefficient exceeds the magnitude of the optimal multipliers.
- **Cleanup** (Lemma 6.6): an index j with $z^1 = 0$ and $x_j^1 \neq 0$ corresponds to an artificial variable remaining in the basis of the dual problem at value 0, and Lemma 6.6 corresponds to the operation of driving it out of the basis. The textbook procedure pivots on any row with a nonzero entry and may therefore lose optimality of the dual problem (feasibility of the primal problem), whereas the primal ratio test of Lemma 6.6 selects the row so as to preserve it, and corresponds to the dual push of crossover (Remark 6.9).
- **No return to unbounded bounds** (Section 6.4): reaching $x_N^1 = 0$ corresponds to all artificial variables having left the basis of the dual problem, after which the iterations are the primal simplex method for the original dual problem, that is, the classical dual simplex method for the original problem (Remark 6.11).

8.2 The two-phase method for the dual problem

We view the decomposition of Section 7 from the dual side. We continue to denote the dual variables by κ, ν . The dual problem of the slope problem (7) is

$$\max - \left(\sum_{j \in J^-} \kappa_j + \sum_{j \in J^+} \nu_j \right) \quad \text{s.t.} \quad A^\top y + \kappa - \nu = c, \quad \kappa, \nu \geq 0 \quad (10)$$

(since $b = 0$ and the finite bounds are 0, only the dual variables corresponding to unbounded sides appear in the objective). This is the problem obtained from (9) by keeping only the penalty

term, that is, exactly the *Phase 1 for the dual problem*, which introduces artificial variables κ_j ($j \in J^-$), ν_j ($j \in J^+$) into the original dual problem and minimizes their sum. Its optimal value equals z^1 .

On the other hand, the dual problem of the intercept problem is

$$\begin{aligned} \max \quad & b^\top y + \sum_j l_j^B \kappa_j - \sum_j u_j^B \nu_j \\ \text{s.t.} \quad & A^\top y + \kappa - \nu = c, \quad \kappa, \nu \geq 0, \quad \kappa_j = 0 \ (x_j^1 > l_j^1), \quad \nu_j = 0 \ (x_j^1 < u_j^1) \end{aligned}$$

Since no multiplier corresponds to a bound that does not exist in the primal problem, the dual variables on the sides where the bound is infinite are fixed to 0. That is, removing bounds in the intercept problem corresponds to fixing the corresponding multipliers to 0 in the dual problem.

Meanwhile, x^1 at the end of Phase A is an optimal solution of the slope problem (7), that is, of the primal problem of (10). By the complementary slackness theorem, once an optimal solution of the primal problem is fixed, a feasible solution of the dual problem is optimal if and only if it is complementary to that fixed optimal solution. Therefore a feasible solution (y, κ, ν) of (10) is optimal if and only if it satisfies complementarity with x^1 , $\kappa_j(x_j^1 - l_j^1) = 0$, $\nu_j(u_j^1 - x_j^1) = 0$, that is,

$$\kappa_j = 0 \ (x_j^1 > l_j^1), \quad \nu_j = 0 \ (x_j^1 < u_j^1)$$

(no condition is imposed for the slack variables s , since $l^1 = u^1 = x^1 = 0$ for them). This coincides index by index with the conditions for the multipliers fixed to 0 in the dual of the intercept problem, and the remaining constraints $A^\top y + \kappa - \nu = c$, $\kappa, \nu \geq 0$ are also common. Therefore the feasible region of the dual of the intercept problem coincides exactly with the *optimal face of the dual Phase 1*. Moreover, the artificial variables corresponding to the bounds originating from artificial bounds ($l_j^B = 0$ if $x_j^1 = l_j^1 = -1$, $u_j^B = 0$ if $x_j^1 = u_j^1 = 1$) remain on this face as variables with cost 0 (on the optimal face, the sum of the artificial variables is fixed at $-z^1$). Hence solving the intercept problem corresponds to maximizing the original dual objective over the optimal face of the dual Phase 1, that is, to the *Phase 2 of the dual problem*. The disappearance of some of the bounds of the original problem in the intercept problem is the dual counterpart of the operation, in the two-phase method, of removing the variables with positive reduced cost after Phase 1 (restricting to the optimal face).

Thus the symbolic artificial bounding method is at the same time the big- M method for the dual problem and the two-phase method for the dual problem, and this correspondence holds at the level of iterations, not at the level of limiting objective values. The classical fact that the big- M method coincides with the two-phase method as $M \rightarrow +\infty$ [8, 9] is usually stated as a comparison of objective values, but in our setting the first and second components of $\succ$ are assigned exactly to Phase A (Phase 1) and Phase B (Phase 2), respectively. The algorithm of Section 9, based on this correspondence, executes the two phases in sequence without using the lexicographic comparison.

9 The slope–intercept two-stage dual simplex method: summary

Integrating all of the above components, we summarize the *slope–intercept two-stage dual simplex method* for the general linear optimization problem (1) as Algorithm 1. The word “dual” in the name refers both to the fact that each phase is a dual simplex method and to the fact that the whole corresponds to a two-phase method for the dual problem (Section 8.2). The tests (a’)(b’) of Section 6.1 defined the iterations by comparing affine functions with the lexicographic order $\succ$ on coefficient pairs, but, as seen in Section 7, the first component (slope) and the second component (intercept) of this comparison do not act simultaneously in the main iterations. Accordingly, in

Algorithm 1 the main iterations are executed not via the lexicographic comparison but as two phases that solve the slope problem (7) and the intercept problem (8) in sequence. The inputs A, b, c, l, u may be the original data before preprocessing, and free variables require no special preprocessing.

After preprocessing (Section 5.1), Algorithm 1 consists of the following two phases and a cleanup performed when necessary.

- **Phase A (slope problem):** starting from the dual feasible initial basis obtained without computation (Section 5.3), run the dual simplex method on the slope problem (7). Only the basic solution x^1 of the slope problem is maintained; the intercept x^0 is not maintained.
- **Phase B (intercept problem):** from the final basis of Phase A, run the ordinary real-valued dual simplex method on the intercept problem (8). At termination, determine whether the original problem is infeasible, unbounded, or has a finite optimal value.
- **Cleanup:** only when there is a finite optimal value and nonbasic variables remain on artificial bounds ($x_N^1 \neq 0$), construct an optimal basic solution free of M by the primal push of Lemma 6.6.

Finally, the inverse shift returns to the original coordinate system. Ties in the ratio test are broken by the lexicographic rule based on cost perturbation in Section 6.5. Since the problems solved in Phases A and B are both ordinary LPs, in an implementation the standard techniques of the dual simplex method, such as leaving-row selection by the steepest-edge or Devex rules and the Bound Flipping Ratio Test, can be applied directly to each phase without any special treatment of M .

Algorithm 1. SLOPEINTERCEPTDUALTWOPHASE(A, b, c, l, u) — the slope–intercept two-stage dual simplex method

Preprocessing

- 1: Compute the shift t_j for each j , and replace $A, b, l, u \leftarrow A, b - At, l - t, u - t$ ($t_j = 0$ for free variables) ▷ Section 5.1
- 2: Determine the slopes l_j^1, u_j^1 of the bounds for each j ($l^1 = u^1 = 0$ for slack variables) ▷ Section 7

Phase A: slope problem (7)

- 3: $B \leftarrow \{n + 1, \dots, n + m\}$ (slack indices); for each $j = 1, \dots, n$, following (4), set $\eta(j) \leftarrow L$ ($c_j > 0$, or $c_j = 0$ and $l_j > -\infty$), Z ($c_j = 0$ and free variable), U (otherwise) ▷ dual feasible without computation, Proposition 5.1
- 4: $x_N^1 \leftarrow$ the nonbasic slopes of Lemma 6.1; $x_B^1 \leftarrow -A_N x_N^1$ ▷ $A_B = I$. Basic solution x^1 of the slope problem, Lemma 7.1
- 5: **loop**
- 6: For each row i , $v_i^{-,1} \leftarrow l_{\beta(i)}^1 - x_{B,i}^1$, $v_i^{+,1} \leftarrow x_{B,i}^1 - u_{\beta(i)}^1$ ▷ slopes of the violations
- 7: $V_A \leftarrow \{i : v_i^{-,1} > 0 \text{ or } v_i^{+,1} > 0\}$
- 8: **if** $V_A = \emptyset$ **then**
- 9: **break** ▷ optimal basis of the slope problem reached; go to Phase B
- 10: **end if**
- 11: With $w_i^1 \leftarrow \max(v_i^{-,1}, v_i^{+,1})$, $r \leftarrow \arg \max_{i \in V_A} w_i^1 / \sqrt{\gamma_i}$ ▷ Dantzig's rule: $\gamma_i \equiv 1$; ties by an arbitrary deterministic rule
- 12: $\rho \leftarrow +1$ (if $v_r^{-,1} > 0$), -1 (if $v_r^{+,1} > 0$); $\alpha^\top \leftarrow e_r^\top A_B^{-1} A_N$; $\sigma_j, \hat{\alpha}_j, \hat{c}_j \leftarrow \dots$ ▷ Step 2(c)
- 13: $\text{Eligible} \leftarrow \{j \in N : l_j < u_j, \rho \hat{\alpha}_j < 0\}$ ▷ variables finite on both sides are also candidates; never empty
- 14: $q \leftarrow \arg \min_{j \in \text{Eligible}} \hat{c}_j / |\hat{\alpha}_j|$ ▷ status- Z candidates have ratio 0 and take top priority; ties by the rule of Section 6.5

15: Pivot exchanging $\beta(r) \leftrightarrow q$; update B, A_B^{-1}, x^1, η $\triangleright \beta(r)$ goes to the bound on the violated side

16: **end loop**

Phase B: intercept problem (8)

17: $z^1 \leftarrow c^\top x^1$; determine l^B, u^B by (8) $\triangleright x^1$ and z^1 are fixed in Phase B

18: $x_N^0 \leftarrow (l_j^B, u_j^B, 0$ according to status L, U, Z); $x_B^0 \leftarrow A_B^{-1}(b - A_N x_N^0)$ $\triangleright$ one FTRAN

19: **loop** $\triangleright$ dual simplex method of Section 3.1 for (8)

20: $V_B \leftarrow \{i : x_{B,i}^0 < l_{\beta(i)}^B \text{ or } x_{B,i}^0 > u_{\beta(i)}^B\}$

21: **if** $V_B = \emptyset$ **then**

22: **break** $\triangleright$ optimal basis of the intercept problem reached

23: **end if**

24: $r \leftarrow \arg \max_{i \in V_B} \Delta_i / \sqrt{\gamma_i}$ (Δ_i is the violation with respect to l^B, u^B); $\rho \leftarrow$ direction of the violation; $\alpha^\top, \sigma_j, \hat{\alpha}_j, \hat{c}_j \leftarrow \dots$

25: $\text{Eligible} \leftarrow \{j \in N : l_j^B < u_j^B, \rho \hat{\alpha}_j < 0\}$

26: **if** $\text{Eligible} = \emptyset$ **then**

27: **return** “the original problem is infeasible” $\triangleright$ Proposition 9.1(i)

28: **end if**

29: $q \leftarrow \arg \min_{j \in \text{Eligible}} \hat{c}_j / |\hat{\alpha}_j|$; pivot exchanging $\beta(r) \leftrightarrow q$; update B, A_B^{-1}, x^0, η

30: **end loop**

31: **if** $z^1 < 0$ **then**

32: **return** “the original problem is feasible and unbounded” $\triangleright$ Proposition 9.1(ii)

33: **end if**

34: $z^0 \leftarrow c^\top x^0$ $\triangleright$ Proposition 9.1(iii), optimal value

Cleanup

35: Let the basic solution be $x(M) = x^0 + Mx^1$ (x^1 from the end of Phase A, x^0 from the end of Phase B) $\triangleright$ Lemma 7.1

36: **while** there exists a nonbasic variable j with $x_j^1 \neq 0$ **do** $\triangleright$ at most K times

37: Choose one such j , and compute t^* by the primal ratio test of Lemma 6.6 ($\succ$ comparison of affine functions of M)

38: **if** $t^* = M$ **then**

39: Place x_j on the finite side (for a free variable, status Z , value 0) $\triangleright$ basis unchanged, Remark 6.7

40: **else**

41: Pivot exchanging $\beta(r) \leftrightarrow j$ at the row r with $t^* = t_r$; fix $\beta(r)$ at the finite bound reached

42: **end if**

43: Update x^0, x^1, η (and also B, A_B^{-1} if a pivot was performed) $\triangleright$ objective value, primal feasibility, and dual feasibility all unchanged

44: **end while**

45: $x^{**} \leftarrow$ the resulting basic solution x^0 (free of M since $x^1 = 0$) $\triangleright$ optimal basic solution of the original problem (in the preprocessed coordinates)

46: **return** $(x^{**} + t, z^0 + c^\top t)$ $\triangleright$ inverse shift; optimal solution and optimal value in the original coordinates

Correctness. The correctness of Algorithm 1 can be shown directly, without relying on the theory of lexicographic comparison of Section 6.1. Let F be the feasible region of the original problem (1) (after preprocessing).

Proposition 9.1 (Correctness of Algorithm 1). *Algorithm 1 terminates after finitely many iterations, and the following hold.*

(i) It returns “infeasible” if and only if $F = \emptyset$.

- (ii) It returns “feasible and unbounded” if and only if $F \neq \emptyset$ and $\inf_{x \in F} c^\top x = -\infty$.
- (iii) Otherwise, the returned $x^{**} + t$ is an optimal basic solution of the original problem, and $z^0 + c^\top t$ is its optimal value.

Proof. Phase A. The slope problem is an LP in which all bounds are finite, and the initial basis is dual feasible (the correspondence between the signs of the reduced costs c and the statuses is the same as in Proposition 5.1). Phase A is the dual simplex method for the slope problem. Variables finite on both sides are fixed to $[0, 0]$ in the slope problem, but including them as candidates in the ratio test is still valid as a dual simplex method, and preserves the dual feasibility of those variables for Phase B and beyond. Under the lexicographic rule based on cost perturbation, Phase A terminates after finitely many iterations, and since the slope problem is feasible at $x^1 = 0$, it reaches an optimal basis at termination. $z^1 = c^\top x^1$ is its optimal value.

Phase B. The reduced costs are determined only by c and the basis B , and the sign conditions for the statuses L, U, Z do not depend on the problem, so the final basis of Phase A is dual feasible for the intercept problem as well. Moreover, each nonbasic variable j satisfies $x_j^1 = l_j^1$ (status L), $x_j^1 = u_j^1$ (status U), or $x_j^1 = 0$ (status Z , in which case j is a free variable), so by the definition (8), j lies on a finite bound l_j^B or u_j^B of the intercept problem, and a variable in status Z is a free variable in the intercept problem as well. Therefore Phase B can be executed as an ordinary dual simplex method for the intercept problem, and it terminates after finitely many iterations, either with “infeasible” or at an optimal basis.

Relation between the intercept problem and the original problem. At the end of Phase A, x^1 is a feasible solution of the slope problem, so $Ax^1 = 0$ and $l^1 \leq x^1 \leq u^1$. (a) If x^0 is a feasible solution of the intercept problem, then $x := x^0 + Mx^1 \in F$ for sufficiently large M . Indeed, $Ax = b$; in components with $x_j^1 > l_j^1$, x_j satisfies the lower bound (l_j if finite, no constraint otherwise) as $M \rightarrow \infty$; in components with $x_j^1 = l_j^1 = -1$, $x_j^0 \geq 0$ only gives $x_j \geq -M$, but the lower bound of the original problem is $-\infty$; and in components with $x_j^1 = l_j^1 = 0$, $x_j = x_j^0 \geq l_j$ (similarly for upper bounds). (b) Conversely, if $x \in F$, then $x^0 := x - Mx^1$ is a feasible solution of the intercept problem for sufficiently large M . Indeed, $Ax^0 = b$; in components with $x_j^1 = l_j^1 = -1$, $x_j^0 = x_j + M \geq 0$; in components with $x_j^1 = l_j^1 = 0$, $x_j^0 = x_j \geq l_j$; and components with $x_j^1 > l_j^1$ have no lower bound (similarly for upper bounds). In either case $c^\top x = c^\top x^0 + Mz^1$.

(i): By (a)(b), infeasibility of the intercept problem is equivalent to $F = \emptyset$. (ii)(iii): When Phase B reaches an optimal basis, $F \neq \emptyset$. If $z^1 < 0$, the original problem is unbounded by Corollary 7.3(i). If $z^1 = 0$, then by (b), $c^\top x = c^\top(x - Mx^1) \geq z^0$ for any $x \in F$, and by (a), $x^0 + Mx^1 \in F$ attains this value z^0 , so z^0 is the optimal value of the original problem. In this case the basic solution $x(M) = x^0 + Mx^1$ satisfies the bounds of the original problem for all sufficiently large M , and the state is dual feasible for the original problem, so the assumptions of Lemma 6.6 are satisfied, and the cleanup yields an optimal basic solution x^{**} of the original problem while preserving the objective value z^0 . The final inverse shift is justified by Section 5.1. $\square$

On the other hand, by Section 7, each iteration of Phases A and B is also a valid iteration of the dual simplex method for (3) with sufficiently large M (since ties in the slopes are broken arbitrarily in Phase A, exact agreement with the iteration sequence for a numerical M is in general lost; Section 7.4). That is, Algorithm 1 executes the iterations of the artificial bounding method in the limit $M \rightarrow +\infty$, split into the two phases of the slope problem and the intercept problem. Note that when $z^1 = 0$, the existing repositioning [13, 16] may also be used instead of the cleanup (Section 7.5).

The main iterations of Phases A and B do not use the lexicographic comparison $\succ$ of affine functions at all. Phase A compares only slopes (real values with respect to bounds with integer coefficients $0, \pm 1$), and Phase B compares only intercepts (ordinary real values). $\succ$ remains only

in the primal ratio test of the cleanup, where both components are essentially needed, e.g., when the slope of t_i coincides with that of M (Remark 7.8).

Remark 9.2 (Comparison with execution by lexicographic comparison). We compare Algorithm 1 with executing the main iterations as a single sequence of iterations using the lexicographic comparison of $(a')(b')$. The main computations per iteration (BTRAN, FTRAN, computation of the pivot row $\alpha^\top$) are unchanged, and the number of iterations is the same except for differences in tie-breaking, so the reduction in computational cost is small. On the other hand, there are the following advantages (their effect on computing time has not been measured separately).

- (a) **Early switch to real arithmetic:** Phase B is an ordinary real-valued dual simplex method that treats the bounds on sides with $v_i^{\pm,1} < 0$ as nonexistent, tests sides with $v_i^{\pm,1} = 0$ by constant violations, and does not maintain pairs of affine functions. The switching point of Remark 6.11 (reaching $x_N^1 = 0$) is moved forward to the end of Phase A, and this difference is large for problems that terminate with $x_N^1 \neq 0$ (unbounded cases and cases requiring cleanup). In Phase A the intercept x^0 need not be updated either; it suffices to compute it with one FTRAN at the transition to Phase B.
- (b) **Numerical robustness:** the floating-point equality test “are the slopes equal” between different rows, which was needed in the lexicographic comparison $\succ$, becomes unnecessary; what remains is only the sign test of w_i^1 for each row (of the same kind as the usual primal feasibility tolerance). Moreover, (7) has right-hand side 0 and bounds only $0, \pm 1$, and is not affected by the scale of b .

As a concern, the effect on the number of iterations of losing the discrimination of tied slopes by intercepts in Phase A needs to be checked empirically.

10 Numerical experiments

We compare ENOMOTO-Solver (implemented in Rust, with a Python interface), a solver implementing Algorithm 1, with the dual simplex method of the open-source LP solver HiGHS[26] on a total of 120 problems from Netlib, Kennington, and Mittelman. The aim of this section is to show that the slope–intercept two-stage dual simplex method runs at a practical speed when combined with standard techniques such as the steepest-edge rule, BFRT, and the Forrest–Tomlin update.

10.1 Overview of the implementation

ENOMOTO-Solver proceeds as follows.

- (1) **Presolve:** scaling by Ruiz equilibration[31], removal of duplicate and dominated rows, bound propagation, row and column singletons, equality doubletons, dual fixing, substitution of implied free columns, merging parallel columns, and so on, repeated until a fixed point is reached.
- (2) **Conversion to standard form:** we apply the shift of Section 5.1 and attach a zero-fixed slack to each row. Infinite bounds are kept infinite rather than replaced by a numerical M . When the constraint matrix splits into several connected components, each component is solved separately.
- (3) **Phase A, Phase B, and cleanup:** executed as in Algorithm 1. If no variable is placed on an infinite side in the initial configuration ($S = \emptyset$), Phase A is skipped. The leaving row is chosen by the dual steepest-edge rule[5], the ratio test is BFRT, and the basis matrix is factorized by sparse LU factorization with the Markowitz rule[30] and the Forrest–Tomlin update[29].

- (4) **Final polishing and postsolve:** the resulting basis is checked against the true bounds and costs. Only if dual infeasible columns remain because of floating-point errors do we finish with the primal simplex method. Finally, the presolve is traversed in reverse to recover a solution of the original problem.

In the default setting, if $z^1 < 0$ at the end of Phase A, the solver returns “infeasible or unbounded” and stops (the same treatment as in the existing method of Section 7.5). If the solver is configured to proceed to Phase B, it distinguishes the two cases as in Proposition 9.1.

10.2 Implementation techniques not found in HiGHS

By comparison with the source code of HiGHS 1.15.1, we list the main techniques that are absent from, or treated differently in, the HiGHS dual simplex method.

- (a) **Slope–intercept phase structure:** the dual phase 1 of HiGHS also solves the same auxiliary problem as the slope problem, but differs in that it bounds free variables by $[-1000, 1000]$. Moreover, after phase 1, HiGHS restores the original bounds and performs phase 2, and if the optimal value of phase 1 is negative, it stops without distinguishing infeasibility from unboundedness. Our implementation solves the intercept problem in Phase B and removes variables on artificial bounds in the cleanup.
- (b) **Fused FTRAN:** the three FTRANs needed in one iteration (the entering column, the vector used to update the dual steepest-edge weights, and the contribution of the variables flipped by BFRT) are solved together in a single pass over the LU factors. HiGHS solves them separately.
- (c) **Bordered LU factorization:** nearly dense columns are separated and handled separately as a Schur complement.
- (d) **Reuse of the pivot order:** at refactorization, the column pivot order of the previous factorization is reused as the initial order. In HiGHS, a similar mechanism is used only for hot starts.
- (e) **Refactorization criteria:** every few iterations, the residual $\|A_B x_B - \text{right-hand side}\|$ of the basic solution is checked, and a refactorization is performed when accumulation of errors is detected. In addition, we ported the HiGHS rule that schedules refactorization on the basis of an estimate of the computational cost, and tuned its thresholds on Netlib.

On the other hand, HiGHS also has almost all of the presolve techniques. Parallelization, such as solving connected components in parallel, is either not triggered or shows no effect at the scale of Netlib, so we do not claim it as an effect here.

10.3 Experimental environment and measurement

The computer has an AMD Ryzen 7 5700U (8 cores, 16 threads), 16 GB of memory, and Windows 11. For HiGHS we used the Python binding highspy 1.15.1 with all default settings except for suppressing output (presolve on; the dual simplex method is the serial version). ENOMOTO-Solver was also run with all default settings. The test set consists of 120 problems in total: the 93 LP problems of Netlib[32], 16 problems from the Kennington test set[35], and 11 problems from Mittelmann’s LP benchmarks[36] (LPopt). The 11 Mittelmann problems are those, among the 44 problems of that benchmark, that HiGHS solved within 10 minutes with its default settings.

We measure the wall-clock time of the solve call, excluding reading the MPS file and building the model. For both solvers, the time includes presolve and postsolve. The times of ENOMOTO-Solver are the median of three measurements for Netlib and Kennington, and a single measurement for Mittelmann. A time limit of 1,500 s was set for cont1 and neos, but both were solved

before reaching it. The times of HiGHS were measured on the same computer but at different dates and times from those of ENOMOTO-Solver. On this computer, processing speed varied by a factor of about 1.5–1.8 depending on the time of day, so the ratios to HiGHS below are only indicative. For fome13 from Mittelmann, the time of HiGHS was not measured, so this problem is excluded from the comparison with HiGHS.

ENOMOTO-Solver returned an optimal solution for all 120 problems, and the relative difference from the optimal value of HiGHS, $|z_{\text{ENOMOTO}} - z_{\text{HiGHS}}| / \max(1, |z_{\text{HiGHS}}|)$, was at most 9.8×10^{-11} . On the 119 problems for which HiGHS times are available, ENOMOTO-Solver was faster on 98, the geometric mean of the ratios was 0.58, and the geometric mean shifted by 100 ms was 0.213 s vs. 0.251 s (ratio 0.85).

10.4 Results on Netlib

Table 1 shows the results aggregated by number of nonzeros. ENOMOTO-Solver was faster on 80 of the 93 problems; the geometric mean of the ratios was 0.56, and the geometric mean shifted by 10 ms was 21.7 ms vs. 29.0 ms. On the other hand, the total time, which is dominated by a small number of large problems, is almost the same: 18.79 s vs. 19.22 s.

Table 1: Comparison of computation times on the 93 Netlib problems (ENOMOTO-Solver vs. HiGHS). “Ratio” is the time of ENOMOTO-Solver divided by the time of HiGHS; a value below 1 means that ENOMOTO-Solver is faster. nnz is the number of nonzeros.

Group	Problems	Faster	ENOMOTO [s]	HiGHS [s]	Ratio of totals	Geom. mean of ratios
nnz < 5000	49	48	0.186	0.360	0.52	0.44
$5000 \leq \text{nnz} < 20000$	29	21	1.352	1.595	0.85	0.75
nnz ≥ 20000	15	11	17.249	17.264	1.00	0.71
HiGHS time ≥ 0.1 s	19	12	17.954	18.041	1.00	0.76
All	93	80	18.787	19.219	0.98	0.56

Table 2 lists the 19 problems for which the time of HiGHS is at least 0.1 s. On these 19 problems, ENOMOTO-Solver was faster on 12; the geometric mean of the ratios is 0.76 and the ratio of total times is 1.00. The problems on which it is slower are pilot.ja, d2q06c (1.31–1.38 times), grow22, 25fv47, fit2p (1.14–1.22 times), df001, and degen3 (1.06–1.09 times).

10.5 Comparison on large-scale problems

Table 3 shows the results on the 16 problems of the Kennington test set. Both solvers returned optimal solutions for all 16 problems, and the relative difference of optimal values was at most 3.1×10^{-15} . ENOMOTO-Solver was faster on 13 problems; the geometric mean of the ratios was 0.48, and the total time was 18.01 s vs. 43.61 s. In particular, on the four osa problems the ratios are 0.08–0.22, a large speedup. On the other hand, it is 1.14–1.30 times slower on pds-20, cre-a, and cre-d.

Table 4 shows the results on the 11 problems from Mittelmann’s LP benchmarks. ENOMOTO-Solver returned an optimal solution for all 11 problems, and the relative difference from the optimal value of HiGHS was at most 1.1×10^{-11} . On the 10 problems for which HiGHS times are available, ENOMOTO-Solver was faster on 5; the geometric mean of the ratios was 1.02, and the total time was 1889 s vs. 1669 s (ratio 1.13). The problems on which it is slower are neos, cont1 (1.55–1.57 times), and pds-100 (1.18 times), and the difference in total time arises almost entirely from these three problems.

As shown above, ENOMOTO-Solver runs at a speed comparable to HiGHS on large-scale problems as well. In large, sparse problems, the right-hand sides and solution vectors of FTRAN and BTRAN, as well as the pivot row, become extremely sparse (hyper-sparse), and operations

Table 2: Netlib problems for which the computation time of HiGHS is at least 0.1 s.

Problem	nnz	ENOMOTO [s]	HiGHS [s]	Ratio
df001	35632	7.84	7.17	1.09
pilot87	73152	4.06	4.37	0.93
pilot	43167	0.943	1.15	0.82
fit2p	50284	1.28	1.12	1.14
d2q06c	32417	1.20	0.917	1.31
maros-r7	144848	0.765	0.830	0.92
greenbeb	30877	0.373	0.425	0.88
cycle	20720	0.020	0.275	0.07
greenbea	30877	0.196	0.242	0.81
25fv47	10400	0.226	0.197	1.15
degen3	24646	0.204	0.193	1.06
pilotnov	13057	0.087	0.191	0.45
nesm	13288	0.072	0.171	0.42
fit2d	129018	0.078	0.157	0.50
pilot.we	9126	0.123	0.148	0.83
80bau3b	21002	0.132	0.135	0.98
d6cube	37704	0.075	0.134	0.56
pilot.ja	14698	0.149	0.108	1.38
grow22	8252	0.132	0.108	1.22

Table 3: Comparison of computation times on the Kennington test set. m, n are the numbers of rows and columns of the original problem, and nnz is the number of nonzeros in the constraint matrix. The times of ENOMOTO-Solver are the median of three measurements.

Problem	m	n	nnz	ENOMOTO [s]	HiGHS [s]	Ratio
cre-a	3516	4067	14987	0.091	0.076	1.20
cre-b	9648	72447	256095	1.66	1.72	0.97
cre-c	3068	3678	13244	0.082	0.101	0.81
cre-d	8926	69980	242646	0.864	0.757	1.14
ken-07	2426	3602	8404	0.013	0.026	0.48
ken-11	14694	21349	49058	0.193	0.249	0.78
ken-13	28632	42659	97246	0.912	1.02	0.89
ken-18	105127	154699	358171	7.74	10.77	0.72
osa-07	1118	23949	143694	0.057	0.256	0.22
osa-14	2337	52460	314760	0.177	0.996	0.18
osa-30	4350	100024	600138	0.488	4.36	0.11
osa-60	10280	232966	1397793	1.57	19.25	0.08
pds-02	2953	7535	16390	0.025	0.127	0.19
pds-06	9881	28655	62524	0.209	0.443	0.47
pds-10	16558	48763	106436	0.622	0.923	0.67
pds-20	33874	105728	230200	3.31	2.54	1.30

Table 4: Comparison of computation times on the 11 problems from Mittelman’s LP benchmarks. All times are single measurements. The time of HiGHS on fome13 was not measured.

Problem	ENOMOTO [s]	HiGHS [s]	Ratio
cont1	375.8	242.9	1.55
ex10	83.8	90.4	0.93
fome13	18.5	—	—
irish-electricity	124.9	156.3	0.80
neos	568.6	363.0	1.57
nug08-3rd	261.8	283.9	0.92
pds-100	89.5	75.9	1.18
s250r10	75.2	108.3	0.69
square41	105.5	95.2	1.11
stormG2_1000	69.7	64.0	1.09
supportcase10	134.7	189.2	0.71

that exploit this sparsity strongly affect the computation time[27]. Analyzing the causes of the differences on the problems where our solver is slower is left for future work; such linear algebra techniques are independent of the method, and all of them can be applied as they are to each phase.

Use of generative AI

In this work, we used Claude, a generative AI by Anthropic (used via Claude Code), for the following purposes.

- **Solver implementation:** most of the implementation of ENOMOTO-Solver was carried out by the AI under the author’s design and direction. Many of the implementation techniques listed in Section 10.2 were proposed by the AI, and the author decided whether to adopt them based on the results of comparative experiments.
- **Numerical experiments:** the AI was used to write the measurement scripts, run the measurements, and aggregate the results.
- **Manuscript preparation:** the AI was used for drafting the text of the manuscript, organizing the written proofs, surveying the literature, and revising.

The ideas and theory of the method in this paper (the analysis with a symbolic M , the decomposition into the slope problem and the intercept problem, the distinction of the three cases, and the correspondence between the big- M method and the two-phase method for the dual problem) are due to the author. All text, proofs, code, and numerical results generated by the AI have been checked by the author, and the author takes responsibility for the content of this paper. The source code and the experimental scripts and data are available at https://github.com/enomotokan/enomoto_solver.

References

- [1] G. B. Dantzig, *Linear Programming and Extensions*, Princeton University Press, 1963.
- [2] G. B. Dantzig, A. Orden, P. Wolfe, “The generalized simplex method for minimizing a linear form under linear inequality restraints,” *Pacific Journal of Mathematics*, 5(2), 183–195, 1955.

- [3] C. E. Lemke, “The dual method of solving the linear programming problem,” *Naval Research Logistics Quarterly*, 1(1), 36–47, 1954.
- [4] R. G. Bland, “New finite pivoting rules for the simplex method,” *Mathematics of Operations Research*, 2(2), 103–107, 1977.
- [5] J. J. Forrest, D. Goldfarb, “Steepest-edge simplex algorithms for linear programming,” *Mathematical Programming*, 57(1–3), 341–374, 1992.
- [6] P. M. J. Harris, “Pivot selection methods of the Devex LP code,” *Mathematical Programming*, 5(1), 1–28, 1973.
- [7] I. Maros, *Computational Techniques of the Simplex Method*, Springer, 2003.
- [8] V. Chvátal, *Linear Programming*, W. H. Freeman, 1983.
- [9] D. Bertsimas, J. N. Tsitsiklis, *Introduction to Linear Optimization*, Athena Scientific, 1997.
- [10] H. D. Sherali, A. L. Soyster, “Preemptive and nonpreemptive multi-objective programming: Relationship and counterexamples,” *Journal of Optimization Theory and Applications*, 39(2), 173–186, 1983.
- [11] M. Cococcioni, L. Fiaschi, “The Big-M method with the numerical infinite M,” *Optimization Letters*, 15(7), 2455–2468, 2021.
- [12] P. Wolfe, “The composite simplex algorithm,” *SIAM Review*, 7(1), 42–54, 1965.
- [13] R. Fourer, “Notes on the dual simplex method,” Draft report, Northwestern University, 1994.
- [14] E. Kostina, “The long step rule in the bounded-variable dual simplex method: Numerical experiments,” *Mathematical Methods of Operations Research*, 55(3), 413–429, 2002.
- [15] I. Maros, “A piecewise linear dual phase-1 algorithm for the simplex method,” *Computational Optimization and Applications*, 26(1), 63–81, 2003.
- [16] A. Koberstein, *The Dual Simplex Method, Techniques for a Fast and Stable Implementation*, Dissertation, Universität Paderborn, 2005.
- [17] R. E. Bixby, “Solving real-world linear programs: A decade and more of progress,” *Operations Research*, 50(1), 3–15, 2002.
- [18] A. Koberstein, U. H. Suhl, “Progress in the dual simplex method for large scale LP problems: practical dual phase 1 algorithms,” *Computational Optimization and Applications*, 37(1), 49–65, 2007.
- [19] M. Huang, Y. Zhong, H. Yang, J. Wang, F. Zhang, B. Bai, L. Shi, “Simplex initialization: A survey of techniques and trends,” arXiv:2111.03376, 2021.
- [20] N. Megiddo, “On finding primal- and dual-optimal bases,” *ORSA Journal on Computing*, 3(1), 63–65, 1991.
- [21] R. E. Bixby, M. J. Saltzman, “Recovering an optimal LP basis from an interior point solution,” *Operations Research Letters*, 15, 169–178, 1994.
- [22] S. Zionts, “The criss-cross method for solving linear programming problems,” *Management Science*, 15(7), 426–445, 1969.
- [23] T. Terlaky, “A convergent criss-cross method,” *Optimization*, 16(5), 683–690, 1985.

- [24] K. Fukuda, T. Terlaky, “Criss-cross methods: A fresh view on pivot algorithms,” *Mathematical Programming*, 79(1–3), 369–395, 1997.
- [25] R. J. Vanderbei, *Linear Programming: Foundations and Extensions*, 5th ed., International Series in Operations Research & Management Science, vol. 285, Springer, 2020.
- [26] Q. Huangfu, J. A. J. Hall, “Parallelizing the dual revised simplex method,” *Mathematical Programming Computation*, 10(1), 119–142, 2018.
- [27] J. A. J. Hall, K. I. M. McKinnon, “Hyper-sparsity in the revised simplex method and how to exploit it,” *Computational Optimization and Applications*, 32(3), 259–283, 2005.
- [28] J. R. Gilbert, T. Peierls, “Sparse partial pivoting in time proportional to arithmetic operations,” *SIAM Journal on Scientific and Statistical Computing*, 9(5), 862–874, 1988.
- [29] J. J. H. Forrest, J. A. Tomlin, “Updated triangular factors of the basis to maintain sparsity in the product form simplex method,” *Mathematical Programming*, 2(1), 263–278, 1972.
- [30] H. M. Markowitz, “The elimination form of the inverse and its application to linear programming,” *Management Science*, 3(3), 255–269, 1957.
- [31] D. Ruiz, “A scaling algorithm to equilibrate both rows and columns norms in matrices,” Technical Report RAL-TR-2001-034, Rutherford Appleton Laboratory, 2001.
- [32] D. M. Gay, “Electronic mail distribution of linear programming test problems,” *Mathematical Programming Society COAL Newsletter*, 13, 10–12, 1985.
- [33] Gurobi Optimization, LLC, *Gurobi Optimizer Reference Manual*, <https://docs.gurobi.com/projects/optimizer/en/current/> (accessed September 25, 2026). See the status codes INF_OR_UNBD and UNBOUNDED, the parameters DualReductions and Method, and the section on Concurrent Optimization.
- [34] IBM, *IBM ILOG CPLEX Optimization Studio 20.1.0 Documentation*, <https://www.ibm.com/docs/en/icos/20.1.0> (accessed September 25, 2026). See the solution status codes (CPX_STAT_INForUNBD) and the parameter LPMETHOD.
- [35] W. J. Carolan, J. E. Hill, J. L. Kennington, S. Niemi, S. J. Wichmann, “An empirical evaluation of the KORBX algorithms for military airlift applications,” *Operations Research*, 38(2), 240–248, 1990.
- [36] H. D. Mittelmann, “Benchmarks for optimization software,” <https://plato.asu.edu/bench.html>.
- [37] E. D. Andersen, Y. Ye, “Combining interior-point and pivoting algorithms for linear programming,” *Management Science*, 42(12), 1719–1731, 1996.